

ARQUITECTURA DE SOLUCIÓN

BANCA POR INTERNET BP

Documento de Arquitectura de Solución

Autor: Gustavo Raza

Rol: Arquitecto de Soluciones – Líder Técnico de Desarrollo

Fecha: 13 de julio de 2026

Versión: 1.0

CONTROL DEL DOCUMENTO

Campo	Valor
Título	Arquitectura de Solución – Banca por Internet BP
Versión	1.0
Estado	Propuesta de arquitectura objetivo
Autor	Gustavo Raza
Rol	Arquitecto de Soluciones – Líder Técnico de Desarrollo
Fecha	13 de julio de 2026
Clasificación	Documento para evaluación técnica

Este documento plantea una arquitectura objetivo para BP. No describe una solución ya implementada y tampoco reemplaza las validaciones institucionales, regulatorias, operativas y de capacidad que deberán completarse antes de iniciar su ejecución.

Tabla de contenido

CONTROL DEL DOCUMENTO	2
1. RESUMEN EJECUTIVO.....	10
1.1 Propósito del documento	10
1.2 Contexto de la solución	10
1.3 Objetivos arquitectónicos	10
1.4 Resumen de la solución propuesta	10
2. ALCANCE Y CONTEXTO	12
2.1 Alcance funcional	12
2.2 Alcance técnico	12
2.3 Elementos fuera de alcance	12
2.4 Actores involucrados	12
2.5 Sistemas internos y externos	12
2.6 Restricciones	13
2.7 Supuestos arquitectónicos	13
2.8 Información pendiente de validación	13
3. REQUISITOS Y ATRIBUTOS DE CALIDAD	14
3.1 Requisitos funcionales	14
3.2 Requisitos no funcionales	14
3.3 Seguridad	14
3.4 Alta disponibilidad	14
3.5 Tolerancia a fallos	14
3.6 Recuperación ante desastres	14
3.7 Rendimiento y escalabilidad	15
3.8 Observabilidad	15
3.9 Excelencia operativa	15
3.10 Trazabilidad con los criterios de evaluación	15
4. PRINCIPIOS Y DECISIONES ARQUITECTÓNICAS	16
4.1 Principios de arquitectura	16
4.2 Arquitectura de microservicios pragmáticos	16

4.3 Desacoplamiento, cohesión y reutilización	16
4.4 Microsoft Azure como plataforma cloud.....	16
4.5 Plataforma de ejecución	17
4.6 Integración síncrona y asíncrona.....	17
4.7 Persistencia y propiedad de los datos	17
4.8 Decisiones de seguridad.....	17
4.9 Decisiones de resiliencia.....	17
4.10 Alternativas evaluadas y trade-offs	18
5. ARQUITECTURA C4.....	19
5.1 Modelo C4 utilizado.....	19
5.2 Diagrama C4 de Contexto	19
5.3 Explicación del diagrama de Contexto.....	19
5.4 Diagrama C4 de Contenedores	20
5.5 Explicación del diagrama de Contenedores.....	20
5.6 Diagrama C4 de Componentes.....	21
5.7 Explicación del diagrama de Componentes	21
6. ARQUITECTURA DE CANALES Y EXPERIENCIA DIGITAL.....	22
6.1 Aplicación web SPA	22
6.2 Aplicación móvil multiplataforma	22
6.3 BFF Web	22
6.4 BFF Mobile.....	22
6.5 API Gateway.....	22
6.6 Gestión de sesiones y clientes frecuentes	23
7. AUTENTICACIÓN, IDENTIDAD Y ONBOARDING.....	24
7.1 Arquitectura de identidad	24
7.2 OAuth 2.0 y OpenID Connect	24
7.3 Authorization Code Flow con PKCE	24
7.4 Autenticación multifactor	24
7.5 Step-up authentication	24
7.6 Biometría local del dispositivo	25
7.7 Passkeys y FIDO2	25
7.8 Onboarding con reconocimiento facial	25

7.9 Gestión de secretos e identidades administradas	26
8. SERVICIOS DE NEGOCIO	28
8.1 Perfil del cliente	28
8.2 Productos.....	28
8.3 Movimientos	28
8.4 Transferencias	28
8.5 Pagos	28
8.6 Beneficiarios.....	28
8.7 Límites y reglas transaccionales.....	29
8.8 Antifraude	29
8.9 Onboarding	29
8.10 Notificaciones	29
8.11 Auditoría	29
8.12 Conciliación.....	29
9. INTEGRACIÓN CON SISTEMAS EMPRESARIALES.....	30
9.1 Integración con el Core bancario.....	30
9.2 Integración con el sistema complementario del cliente.....	30
9.3 Anti-Corruption Layer	30
9.4 Integración mediante API Gateway	30
9.5 Comunicación REST	30
9.6 Mensajería con Azure Service Bus	30
9.7 Gestión de errores de integración	31
9.8 Timeouts, reintentos y circuit breakers	31
10. PROCESAMIENTO TRANSACCIONAL	32
10.1 Transferencias entre cuentas propias	32
10.2 Transferencias interbancarias	32
10.3 Pagos.....	32
10.4 Saga orquestada.....	32
10.5 Idempotencia.....	32
10.6 Transactional Outbox	32
10.7 Conciliación.....	33
10.8 Consistencia y manejo de transacciones distribuidas	33

11. ARQUITECTURA DE DATOS	35
11.1 Principios de persistencia	35
11.2 Base de datos por servicio.....	35
11.3 Azure Database for PostgreSQL.....	35
11.4 Azure Cache for Redis	35
11.5 Patrón Cache-Aside.....	35
11.6 CQRS ligero	35
11.7 Optimización para clientes frecuentes	36
11.8 Protección y clasificación de la información	36
11.9 Retención y ciclo de vida de los datos	36
12. AUDITORÍA Y NOTIFICACIONES.....	37
12.1 Auditoría de acciones del cliente.....	37
12.2 Separación entre auditoría y logs operativos.....	37
12.3 Inmutabilidad e integridad de registros	37
12.4 Eventos de auditoría	37
12.5 Notificaciones asíncronas.....	37
12.6 Notificaciones push	37
12.7 Correo electrónico	38
12.8 SMS	38
12.9 Reintentos y manejo de entregas fallidas	38
13. SEGURIDAD.....	39
13.1 Modelo de seguridad por capas.....	39
13.2 Protección perimetral	39
13.3 Web Application Firewall.....	39
13.4 Protección DDoS	39
13.5 Gestión de identidades y accesos.....	39
13.6 Protección de APIs	39
13.7 Cifrado en tránsito y en reposo	40
13.8 Azure Key Vault.....	40
13.9 Managed Identities	40
13.10 Seguridad de contenedores.....	40
13.11 Seguridad de datos	40

13.12 Prevención y detección de fraude	40
13.13 Trazabilidad y no repudio	40
14. DISPONIBILIDAD, RESILIENCIA Y RECUPERACIÓN.....	41
14.1 Alta disponibilidad multizona	41
14.2 Tolerancia a fallos.....	41
14.3 Auto-healing.....	41
14.4 Escalabilidad automática.....	41
14.5 Región secundaria warm standby	41
14.6 Recuperación ante desastres	41
14.7 RTO y RPO	42
14.8 Estrategia de respaldos.....	42
14.9 Pruebas de recuperación.....	42
14.10 Degradación controlada del servicio	42
15. ARQUITECTURA DE DESPLIEGUE EN AZURE	43
15.1 Vista general de despliegue	43
15.2 Azure Kubernetes Service	43
15.3 Azure Functions	44
15.4 Servicios de integración.....	44
15.5 Servicios de datos.....	44
15.6 Servicios de seguridad.....	44
15.7 Servicios de observabilidad.....	44
15.8 Distribución regional y por zonas	44
15.9 Ambientes de desarrollo, pruebas y producción	45
16. OBSERVABILIDAD Y OPERACIÓN	46
16.1 Estrategia de observabilidad	46
16.2 Logs	46
16.3 Métricas	46
16.4 Trazas distribuidas.....	46
16.5 OpenTelemetry	46
16.6 Correlation ID	46
16.7 Alertas	46
16.8 Tableros operativos	46

16.9 Monitoreo de experiencia del usuario	47
16.10 Gestión de incidentes	47
17. DEVOPS, CALIDAD Y EXCELENCIA OPERATIVA	48
17.1 Estrategia de integración y entrega continua.....	48
17.2 Infraestructura como Código	48
17.3 Gestión de configuraciones.....	48
17.4 Gestión de secretos.....	48
17.5 Estrategia de pruebas	48
17.6 Pruebas de seguridad	48
17.7 Pruebas de rendimiento	48
17.8 Pruebas de resiliencia	48
17.9 Estrategia de despliegue	49
17.10 Rollback	49
17.11 Gobierno y control de cambios	49
18. REGULACIÓN Y CUMPLIMIENTO.....	50
18.1 Consideraciones regulatorias	50
18.2 Protección de datos personales	50
18.3 Trazabilidad de operaciones.....	50
18.4 Segregación de funciones	50
18.5 Retención de registros.....	50
18.6 Gestión de evidencias	50
18.7 Validaciones pendientes con las áreas de cumplimiento.....	50
19. COSTOS Y OPTIMIZACIÓN.....	51
19.1 Principales generadores de costo.....	51
19.2 Estrategia de dimensionamiento	51
19.3 Escalabilidad según demanda.....	51
19.4 Optimización de recursos.....	51
19.5 Ambientes no productivos	51
19.6 Gobierno financiero cloud.....	51
19.7 Consideraciones de crecimiento	51
20. RIESGOS ARQUITECTÓNICOS.....	52
20.1 Riesgos técnicos.....	52

20.2 Riesgos de integración.....	52
20.3 Riesgos de seguridad	52
20.4 Riesgos operativos.....	52
20.5 Riesgos regulatorios.....	52
20.6 Riesgos de costos.....	52
20.7 Acciones de mitigación.....	52
21. ROADMAP DE IMPLEMENTACIÓN.....	53
21.1 Criterios de priorización.....	53
21.2 Producto mínimo viable.....	53
21.3 Primera fase de evolución.....	53
21.4 Segunda fase de evolución	53
21.5 Capacidades futuras	53
21.6 Dependencias y condiciones de implementación	54
22. CONCLUSIONES.....	55
22.1 Cumplimiento de los objetivos	55
22.2 Beneficios de la arquitectura	55
22.3 Decisiones que requieren validación	55
22.4 Recomendación final.....	55

1. RESUMEN EJECUTIVO

1.1 Propósito del documento

Este documento define la arquitectura objetivo del sistema de banca por internet de BP. Para ello, describe los componentes principales, las integraciones, los controles de seguridad, los mecanismos de resiliencia, la estrategia de datos y los lineamientos de operación. El resultado busca responder de forma directa al ejercicio y, al mismo tiempo, dejar una base técnica útil para el refinamiento, la estimación, la construcción y la transición a una ejecución.

El nivel de detalle se ha definido para que la propuesta pueda ser revisada tanto por perfiles técnicos como ejecutivos, sin convertirla en una especificación de implementación. Esa distinción es importante. Las decisiones se apoyan en atributos de calidad propios de una plataforma financiera: seguridad, disponibilidad, trazabilidad, consistencia transaccional, capacidad de recuperación, observabilidad y evolución controlada.

1.2 Contexto de la solución

BP necesita ofrecer a sus clientes una experiencia digital consistente desde una aplicación web SPA y una aplicación móvil multiplataforma. Desde ambos canales se deberá consultar información del cliente, productos e histórico de movimientos; también será posible ejecutar transferencias propias e interbancarias, realizar pagos, administrar beneficiarios y recibir notificaciones por distintos mecanismos.

La solución se integra con el Core bancario, que conserva su papel como sistema oficial de registro financiero, y con un sistema complementario que aporta información detallada del cliente. A esto se suman el proveedor corporativo de identidad, el proveedor especializado de verificación biométrica, la red o plataforma interbancaria y los proveedores externos de notificaciones. Cada integración cumple una responsabilidad concreta y debe mantenerse claramente delimitada.

1.3 Objetivos arquitectónicos

- Proteger la información y las operaciones del cliente mediante seguridad por capas y autenticación moderna.
- Mantener como criterio base al Core bancario como sistema oficial de registro de cuentas, saldos y operaciones financieras.
- Considerar de forma explícita desacoplar canales, servicios de negocio e integraciones para facilitar evolución, reutilización y mantenimiento.
- Controlar fallas parciales, duplicidad de solicitudes y consistencia de procesos distribuidos.
- Proporcionar alta disponibilidad multizona y una estrategia regional de recuperación ante desastres.
- Mantener como criterio: garantizar trazabilidad funcional y técnica mediante auditoría, Correlation ID, métricas, logs y trazas distribuidas.
- Considerar de forma explícita aplicar un enfoque de microservicios pragmáticos, evitando complejidad innecesaria en dominios que no la requieren.

1.4 Resumen de la solución propuesta

La propuesta se construye sobre Microsoft Azure. Azure Front Door y Web Application Firewall protegen el ingreso desde los canales; el API Gateway concentra las políticas de exposición, validación y control; y los BFF Web y Mobile ajustan las respuestas a las necesidades de cada experiencia. Los microservicios críticos se ejecutan en Azure Kubernetes Service. Para procesos breves, programados o asíncronos, Azure Functions puede utilizarse cuando aporte una ventaja operativa real.

Las integraciones síncronas utilizan REST. Para la mensajería transaccional y el procesamiento desacoplado se utiliza Azure Service Bus. En transferencias se aplican Saga orquestada, Idempotency Key y Transactional

Outbox, porque allí el control de duplicidad y de fallas parciales no puede quedar implícito. Azure Database for PostgreSQL almacena datos propios de los dominios digitales y estados operativos, mientras que Azure Cache for Redis se mantiene únicamente como una capa de optimización mediante Cache-Aside.

La identidad se delega al producto corporativo mediante OAuth 2.0 y OpenID Connect. Para los canales se recomienda Authorization Code Flow con PKCE, complementado con MFA y step-up authentication cuando la operación sea sensible. El onboarding móvil incorpora reconocimiento facial y prueba de vida a través de un proveedor especializado. Considerar que la biometría local protege el acceso a credenciales del dispositivo, pero no reemplaza al proveedor corporativo de identidad.

La arquitectura contempla alta disponibilidad multizona y una región secundaria en modalidad warm standby. Como punto de partida se asumen un RTO de 60 minutos y un RPO de 15 minutos. Son valores razonables para orientar el diseño, pero todavía deben validarse junto con los volúmenes, la regulación, los costos y las capacidades reales de los sistemas existentes.

2. ALCANCE Y CONTEXTO

2.1 Alcance funcional

- Considerar de forma explícita la consulta de información del cliente, productos, saldos e histórico de movimientos.
- Transferencias entre cuentas propias y transferencias interbancarias.
- Pagos y gestión de beneficiarios.
- Onboarding móvil con verificación facial y prueba de vida.
- Considerar de forma explícita el acceso posterior mediante credenciales corporativas, MFA, biometría local u otros métodos aprobados.
- Notificaciones de movimientos mediante push, correo electrónico y SMS.
- Auditoría de las acciones relevantes ejecutadas por el cliente.

2.2 Alcance técnico

En el escenario planteado, el alcance técnico comprende la arquitectura de canales, seguridad perimetral. BFF, API Gateway, microservicios, mensajería, integración con sistemas empresariales, persistencia de datos de dominios digitales, caché, auditoría, notificaciones, observabilidad, alta disponibilidad, recuperación ante desastres y lineamientos de DevOps y operación.

Se define la arquitectura objetivo y sus decisiones principales, pero no se entregan código fuente, pipelines implementados, Terraform, Bicep ni plantillas de infraestructura. La Infraestructura como Código se mantiene como práctica corporativa recomendada para la fase de implementación.

2.3 Elementos fuera de alcance

- Reemplazo o rediseño interno del Core bancario.
- Considerar de forma explícita la selección contractual de un proveedor corporativo de identidad o del proveedor biométrico.
- Diseño físico detallado de redes, direccionamiento y conectividad híbrida.
- Dimensionamiento final, licenciamiento y estimación contractual de costos.
- Mantener como criterio: implementación de código, infraestructura, automatizaciones o pipelines.
- Considerar de forma explícita la definición jurídica final de políticas de privacidad, retención y residencia de datos.

2.4 Actores involucrados

Actor	Responsabilidad o interacción
Cliente BP	Utiliza los canales digitales para consultar y ejecutar operaciones.
Operaciones y Soporte BP	Supervisa incidentes, conciliaciones, alertas y atención operativa.
Seguridad y Riesgos	Valida controles de identidad, fraude, acceso, cifrado y monitoreo.
Cumplimiento y Legal	Define obligaciones regulatorias, privacidad, evidencia y retención.
Equipos de Desarrollo y Plataforma	Construyen, despliegan y operan los componentes de la solución.
Propietarios de sistemas empresariales	Gestionan el Core, sistema complementario e integraciones internas.

2.5 Sistemas internos y externos

Sistema	Rol
Core bancario	Sistema oficial de registro de cuentas, saldos, movimientos y operaciones financieras.
Sistema complementario del cliente	Aporta información detallada no disponible en el Core.
Proveedor corporativo de identidad	Autentica al cliente y emite tokens mediante OAuth 2.0 y OpenID Connect.

Red o plataforma interbancaria	Procesa o enruta transferencias hacia otras entidades.
Proveedor de verificación biométrica	Ejecuta reconocimiento facial y prueba de vida durante onboarding.
Proveedores de notificaciones	Entregan mensajes push, correo electrónico y SMS.

2.6 Restricciones

- El producto corporativo de identidad ya existe y debe utilizarse.
- El Core bancario conserva el registro oficial financiero.
- Se considerar de forma explícita que la solución debe implementarse sobre una nube pública aprobada; se selecciona Microsoft Azure.
- La arquitectura debe soportar canales web y móvil, alta disponibilidad y recuperación ante desastres.
- Asegurar las capacidades reales de integración de sistemas existentes pueden limitar contratos, latencia y disponibilidad.
- Mantener como criterio: la propuesta debe mantenerse independiente de una implementación específica de código o infraestructura.

2.7 Supuestos arquitectónicos

- Los sistemas empresariales dispondrán de interfaces seguras o de mecanismos de integración que puedan encapsularse mediante la Capa Anticorrupción.
- Asegurar el proveedor corporativo de identidad soportará OAuth 2.0, OpenID Connect, PKCE y MFA.
- Mantener como criterio: la región primaria de Azure dispondrá de zonas de disponibilidad para los servicios seleccionados.
- Se considera de forma explícita que la región secundaria podrá operar en modalidad warm standby con capacidad mínima preparada.
- El RTO inicial de 60 minutos y RPO inicial de 15 minutos son supuestos sujetos a validación institucional.
- Asegurar los volúmenes y picos transaccionales serán confirmados mediante pruebas de capacidad.

2.8 Información pendiente de validación

- Considerar de forma explícita un volumen de clientes, concurrencia, transacciones por segundo y estacionalidad.
- SLA y ventanas de mantenimiento del Core, red interbancaria y sistema complementario.
- Requisitos regulatorios aplicables, residencia de datos y períodos de retención.
- Mantener como criterio: capacidades exactas del proveedor de identidad y del proveedor biométrico.
- Objetivos definitivos de RTO y RPO por capacidad de negocio.
- Presupuesto, modelo de soporte, horarios operativos y estrategia de conectividad híbrida.

3. REQUISITOS Y ATRIBUTOS DE CALIDAD

3.1 Requisitos funcionales

Los requisitos funcionales se organizan alrededor de consulta, transacción, identidad, onboarding, auditoría y comunicación. Por eso, cada operación debe exponerse mediante contratos versionados y controles homogéneos de seguridad, trazabilidad e idempotencia cuando corresponda.

Código	Requisito	Capacidad arquitectónica
RF-01	Consultar histórico de movimientos	Servicio de Movimientos, Core y caché controlada.
RF-02	Consultar información del cliente	Servicio de Perfil y sistema complementario.
RF-03	Transferencias entre cuentas propias	Servicio de Transferencias, Saga e integración Core.
RF-04	Transferencias interbancarias	Servicio de Transferencias, Capa Anticorrupción y red interbancaria.
RF-05	Pagos	Servicio de Pagos y validaciones transaccionales.
RF-06	Notificar movimientos	Eventos, Azure Service Bus y servicio de Notificaciones.
RF-07	Onboarding facial	Servicio de Onboarding y proveedor biométrico.
RF-08	Auditar acciones	Eventos de auditoría y almacén con controles de inmutabilidad.

3.2 Requisitos no funcionales

Los requisitos no funcionales condicionan las decisiones de plataforma y diseño. En consecuencia, la arquitectura prioriza disponibilidad, seguridad, recuperación, rendimiento, observabilidad, operabilidad y evolución. Los valores cuantitativos definitivos deben acordarse mediante SLO y presupuestos de error por capacidad de negocio.

3.3 Seguridad

La seguridad se aplica por capas: protección perimetral, control de identidad, autorización en APIs, segmentación, mínimo privilegio, cifrado, gestión centralizada de secretos, seguridad de contenedores, protección de datos, prevención de fraude y trazabilidad. Las operaciones sensibles requieren step-up authentication y controles de riesgo adicionales.

3.4 Alta disponibilidad

La región primaria utiliza despliegue multizona para los componentes críticos. Los servicios deben evitar dependencias de instancia, conservar estado fuera de los procesos y distribuir réplicas entre zonas. La disponibilidad efectiva dependerá también de los SLA y diseños de los sistemas empresariales integrados.

3.5 Tolerancia a fallos

La solución incorpora timeouts, reintentos controlados, circuit breakers, colas, dead-letter queue, idempotencia, degradación controlada y auto-healing. La decisión busca algo concreto: los reintentos no se aplican indiscriminadamente: deben considerar naturaleza de la operación, idempotencia y riesgo de duplicidad.

3.6 Recuperación ante desastres

La estrategia considera una región secundaria warm standby, respaldos, réplica o restauración preparada de datos, configuración reproducible y procedimientos de conmutación. A partir de ahí, el RTO inicial supuesto es de 60 minutos y el RPO inicial supuesto es de 15 minutos, sujetos a pruebas periódicas y aprobación de BP.

3.7 Rendimiento y escalabilidad

Los componentes sin estado escalan horizontalmente según demanda. Azure Cache for Redis reduce lecturas repetitivas de información apta para caché; CQRS ligero permite separar modelos de consulta cuando aporte valor. La optimización debe basarse en mediciones y no en supuestos generales.

3.8 Observabilidad

Logs, métricas y trazas distribuidas se correlacionan mediante Correlation ID y telemetría basada en OpenTelemetry. Los tableros y alertas deben reflejar salud técnica y resultados de negocio, incluyendo latencia, errores, colas, fallas de integración, entregas de notificaciones y tiempos de transferencia.

3.9 Excelencia operativa

La excelencia operativa se apoya en automatización de despliegues, Infraestructura como Código, segregación de ambientes, pruebas, runbooks, gestión de incidentes, revisión postincidente, control de cambios y prácticas de capacidad y costos. La decisión busca algo concreto: la operación debe diseñarse junto con la solución, no añadirse al final.

3.10 Trazabilidad con los criterios de evaluación

Criterio	Respuesta en la arquitectura	Secciones
Arquitectura desacoplada, reusable y cohesionada	BFF, microservicios pragmáticos, contratos y mensajería.	4, 5, 6, 8 y 9
Seguridad	OAuth 2.0, OpenID Connect, PKCE, MFA, WAF, DDoS, Key Vault.	7 y 13
Alta disponibilidad y tolerancia a fallos	Multizona, AKS, patrones de resiliencia y auto-healing.	14 y 15
Recuperación ante desastres	Warm standby, respaldos, RTO/RPO y pruebas.	14
Auditoría y notificaciones	Procesamiento independiente por eventos.	12
Monitoreo y excelencia operativa	OpenTelemetry, alertas, DevOps y runbooks.	16 y 17
Justificación y alternativas	Decisiones, trade-offs, riesgos y roadmap.	4, 19, 20 y 21

4. PRINCIPIOS Y DECISIONES ARQUITECTÓNICAS

4.1 Principios de arquitectura

- Seguridad y privacidad desde el diseño.
- Core bancario como sistema oficial de registro financiero.
- Contratos explícitos y versionados entre capacidades.
- Propiedad de datos por dominio digital.
- Automatización, observabilidad y recuperación como parte del diseño.
- Complejidad proporcional al riesgo y criticidad del servicio.
- Evolución incremental sin acoplar los canales a sistemas heredados.

4.2 Arquitectura de microservicios pragmáticos

Se adopta una arquitectura de microservicios pragmáticos. En la operación diaria, los límites se definen por capacidades de negocio y necesidad real de autonomía, no por dividir cada función en un servicio independiente. Los dominios críticos, con escalabilidad, seguridad o ciclo de cambio diferenciados, pueden aislarse; capacidades pequeñas y estrechamente relacionadas pueden mantenerse juntas para evitar sobrecarga operativa.

Este enfoque permite evolucionar la solución sin imponer el mismo nivel de aislamiento, despliegue o persistencia a todos los servicios. La decisión deberá revisarse con datos de organización de equipos, volumen, criticidad, dependencia y frecuencia de cambio.

Segunda justificación teórica: este enfoque aplica el principio de alta cohesión y bajo acoplamiento, porque concentra reglas relacionadas dentro de límites de dominio y evita dependencias innecesarias entre capacidades. Además, permite que la autonomía de despliegue se reserve para los servicios que realmente la necesitan, reduciendo el costo cognitivo y operativo de una fragmentación excesiva.

4.3 Desacoplamiento, cohesión y reutilización

El desacoplamiento se logra mediante BFF por canal, API Gateway, contratos REST, mensajería y Capa Anticorrupción. La decisión busca algo concreto: la cohesión se mantiene agrupando reglas y datos dentro del dominio que los posee. La reutilización se aplica a capacidades transversales como identidad, observabilidad, auditoría, notificaciones y políticas de API, evitando crear servicios compartidos que concentren lógica de negocio heterogénea.

Segunda justificación teórica: la separación explícita de responsabilidades limita la propagación de cambios y favorece la sustitución de componentes sin afectar al conjunto. Desde el punto de vista del diseño modular, esto mejora mantenibilidad y capacidad de evolución, porque cada dependencia se consume mediante contratos y no mediante conocimiento interno del componente proveedor.

4.4 Microsoft Azure como plataforma cloud

Microsoft Azure se selecciona como plataforma principal por su oferta integrada de cómputo, seguridad, identidad, mensajería, datos, monitoreo y capacidades multizona. A partir de ahí, la elección no elimina la necesidad de validar disponibilidad regional, requisitos regulatorios, competencias del equipo, costos y condiciones contractuales.

Segunda justificación teórica: una plataforma integrada de servicios administrados reduce la carga operativa asociada a disponibilidad, respaldos, parcheo y monitoreo, permitiendo concentrar el esfuerzo en capacidades de negocio. También ofrece dominios de falla por zona y región, lo que facilita diseñar continuidad y recuperación con mecanismos consistentes dentro del mismo marco de gobierno.

4.5 Plataforma de ejecución

Azure Kubernetes Service es la plataforma principal para microservicios críticos que requieren control de despliegue, escalabilidad, aislamiento y operación consistente. Azure Functions se reserva para tareas breves, programadas o asíncronas. Azure Container Apps se mantiene como alternativa evaluada para escenarios con menor complejidad operativa, pero no constituye la decisión principal.

Segunda justificación teórica: la orquestación declarativa de contenedores mantiene un estado deseado, redistribuye cargas y permite escalado horizontal, características necesarias para servicios críticos con demanda variable. La separación de Azure Functions para tareas breves evita convertir procesos event-driven en cargas permanentes y mantiene proporcionalidad entre complejidad operativa y naturaleza del trabajo.

4.6 Integración síncrona y asíncrona

REST se utiliza cuando el consumidor requiere una respuesta inmediata y existe una dependencia funcional explícita. Azure Service Bus se utiliza para eventos y comandos asíncronos que deben procesarse de forma desacoplada, con reintentos, suscripciones independientes y dead-letter queue. La selección depende de semántica, latencia, consistencia y tolerancia a indisponibilidad.

Segunda justificación teórica: combinar interacción síncrona y mensajería asíncrona permite seleccionar el modelo de consistencia según la necesidad del negocio. REST conserva inmediatez cuando la respuesta forma parte del flujo del usuario, mientras la mensajería introduce desacoplamiento temporal y evita que procesos secundarios amplifiquen la latencia o disponibilidad de la operación principal.

4.7 Persistencia y propiedad de los datos

Cada servicio es propietario de sus datos digitales y evita acceso directo a tablas de otros dominios. La decisión busca algo concreto: azure Database for PostgreSQL es la opción principal para persistencia relacional administrada de beneficiarios, onboarding, idempotencia, estados de transferencias, Transactional Outbox, conciliación y otros datos propios. El Core continúa siendo el registro oficial financiero.

Segunda justificación teórica: la propiedad de datos por dominio aplica encapsulamiento también a la persistencia, impidiendo que otros servicios dependan de estructuras internas que pueden cambiar. Esto permite evolucionar esquemas y reglas de integridad de manera independiente, y reduce el riesgo de que una modificación local genere efectos no controlados en toda la plataforma.

4.8 Decisiones de seguridad

La identidad se centraliza en el producto corporativo. Con ese criterio, OAuth 2.0 y OpenID Connect separan autorización y autenticación; Authorization Code Flow con PKCE protege clientes públicos; MFA y step-up authentication elevan la seguridad según el riesgo. Azure Key Vault, Managed Identities, mínimo privilegio y cifrado reducen exposición de secretos y credenciales.

Segunda justificación teórica: centralizar identidad y autorización evita que cada servicio implemente mecanismos propios con niveles de seguridad inconsistentes. A la vez, el uso de tokens de alcance limitado, mínimo privilegio y secretos administrados reduce la superficie de ataque y restringe el impacto de una eventual exposición de credenciales.

4.9 Decisiones de resiliencia

La resiliencia combina prevención, absorción y recuperación: despliegue multizona, timeouts, reintentos limitados, circuit breakers, colas, idempotencia, Saga, Transactional Outbox, conciliación, auto-healing, respaldos y warm standby regional. Ningún patrón elimina por sí solo el riesgo; deben operar en conjunto y probarse.

Segunda justificación teórica: la resiliencia efectiva requiere capas complementarias porque prevención, contención y recuperación resuelven problemas diferentes. Timeouts y circuit breakers limitan fallas en

cascada; idempotencia, Outbox y conciliación preservan consistencia; y alta disponibilidad, respaldos y warm standby permiten recuperar capacidad cuando la falla supera al componente individual.

4.10 Alternativas evaluadas y trade-offs

Decisión	Opción principal	Alternativa	Trade-off principal
Aplicación móvil	React Native	Flutter	React Native facilita reutilización con ecosistema JavaScript. Por esa razón, flutter ofrece UI consistente, pero exige competencias específicas.
Ejecución	Azure Kubernetes Service	Azure Container Apps	AKS aporta control y flexibilidad; aumenta carga operativa. Container Apps simplifica operación, con menor control fino.
Persistencia	Azure Database for PostgreSQL	Otras bases administradas	PostgreSQL ofrece modelo relacional maduro. Esto permite que la selección final depende de estándares y capacidades institucionales.
Integración	REST + Azure Service Bus	Solo REST	La combinación mejora desacoplamiento. En consecuencia, introduce consistencia eventual y operación de mensajería.
Transacciones distribuidas	Saga orquestada	Coreografía	La orquestación hace visible el estado. Dicho de otra forma, concentra coordinación y requiere alta disponibilidad del orquestador.
Recuperación regional	Warm standby	Activo-activo	Warm standby reduce costo y complejidad. A partir de ahí, presenta mayor RTO que activo-activo.
Identidad futura	Passkeys/FIDO2 como evolución	Contraseña permanente	Passkeys reducen phishing, pero requieren adopción, compatibilidad y gobierno.

5. ARQUITECTURA C4

5.1 Modelo C4 utilizado

El modelo C4 comunica la arquitectura en niveles progresivos. El diagrama de Contexto identifica actores y sistemas externos; el diagrama de Contenedores presenta las aplicaciones y plataformas principales; y el diagrama de Componentes profundiza en el Servicio de Transferencias por ser el flujo más crítico y representativo.

Segunda justificación teórica: C4 separa niveles de abstracción para que cada audiencia observe únicamente el detalle que necesita, evitando mezclar contexto, despliegue lógico e implementación interna en una sola vista. Esta progresión mejora trazabilidad y reduce ambigüedad, porque permite relacionar actores, contenedores y componentes sin perder la visión global.

5.2 Diagrama C4 de Contexto

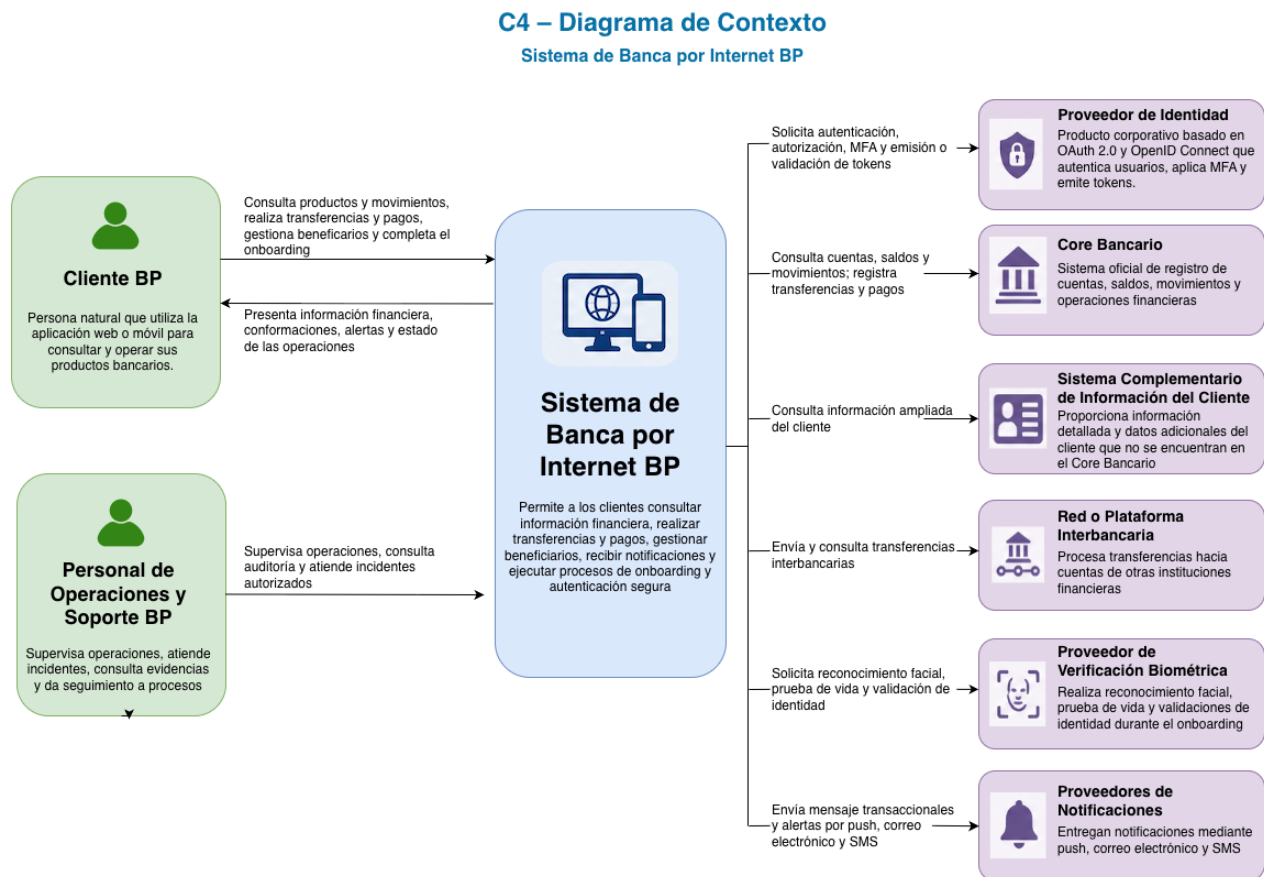


Figura 1. C4 – Diagrama de Contexto de la Banca por Internet BP.

5.3 Explicación del diagrama de Contexto

El Cliente BP accede al sistema de banca por internet para consultar y ejecutar operaciones. Por eso, el personal de Operaciones y Soporte supervisa la plataforma y atiende situaciones operativas. El sistema se integra con el proveedor corporativo de identidad, el Core bancario, el sistema complementario de información del cliente, la red interbancaria, el proveedor biométrico y los proveedores de notificaciones.

El Core se identifica explícitamente como sistema oficial de registro. La decisión busca algo concreto: el proveedor de identidad y el proveedor biométrico cumplen responsabilidades diferentes: el primero autentica y emite tokens; el segundo verifica identidad y prueba de vida durante onboarding.

5.4 Diagrama C4 de Contenedores

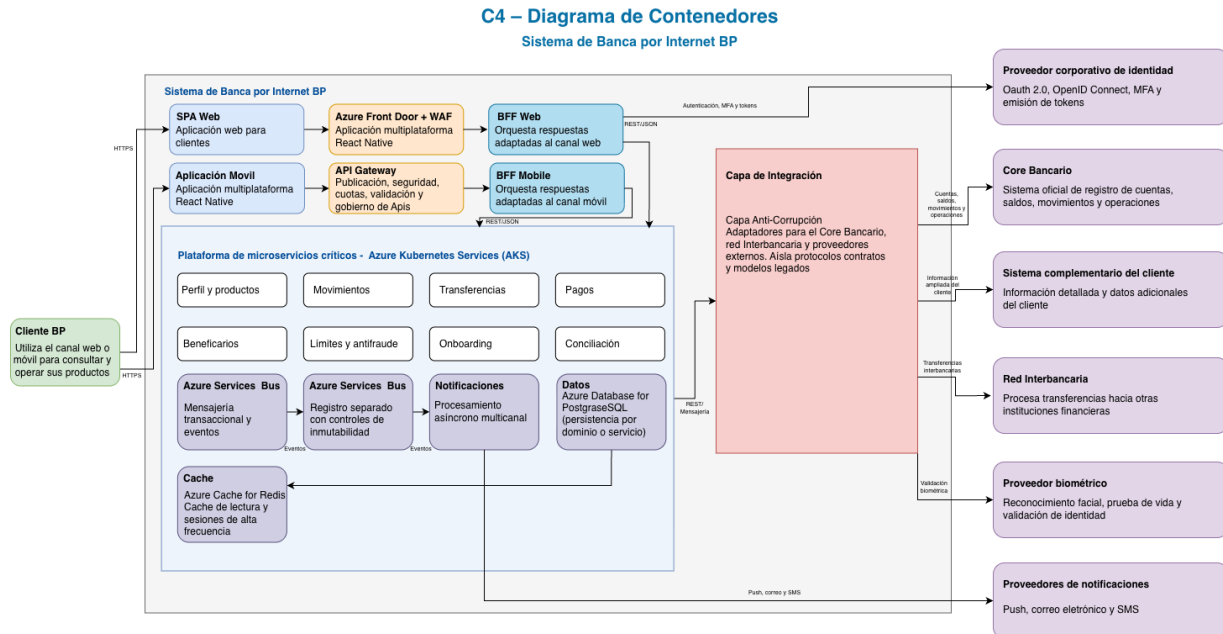


Figura 2. C4 – Diagrama de Contenedores de la Banca por Internet BP.

5.5 Explicación del diagrama de Contenedores

El SPA y la aplicación móvil ingresan por Azure Front Door y WAF, continúan hacia el API Gateway y utilizan BFF independientes. La plataforma de microservicios críticos sobre AKS agrupa servicios de negocio con límites claros. Azure Service Bus desacopla auditoría, notificaciones y procesos posteriores. Azure Database for PostgreSQL almacena datos propios de los dominios digitales y Azure Cache for Redis optimiza consultas frecuentes.

La Capa Anticorrupción protege el modelo interno frente a particularidades de los sistemas empresariales. Por eso, auditoría y Notificaciones consumen eventos de Azure Service Bus de manera independiente; no forman una cadena secuencial ni condicionan la respuesta financiera al cliente.

5.6 Diagrama C4 de Componentes

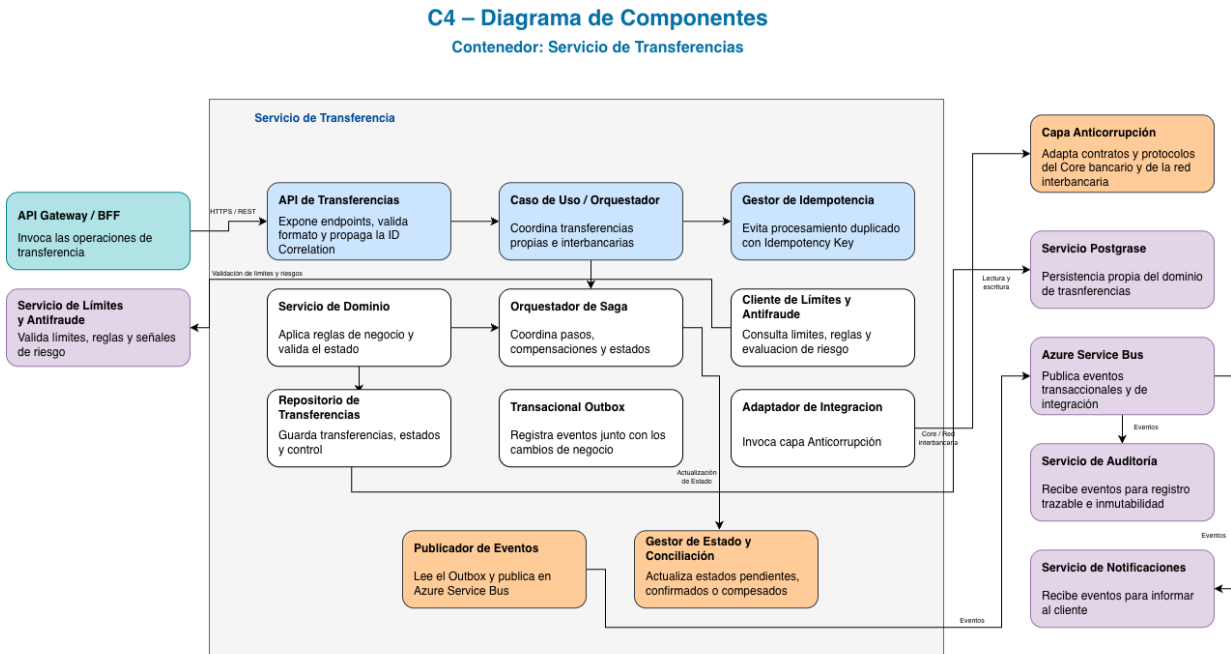


Figura 3. C4 – Componentes del contenedor Servicio de Transferencias.

5.7 Explicación del diagrama de Componentes

El nivel de Componentes se concentra en el contenedor Servicio de Transferencias. En la operación diaria, la API recibe solicitudes; el Caso de Uso u Orquestador coordina validaciones; el Gestor de Idempotencia controla duplicidad; el Servicio de Dominio aplica reglas; el Orquestador de Saga gestiona pasos y compensaciones; y el Adaptador de Integración se comunica con la Capa Anticorrupción.

El Repositorio de Transferencias y el Transactional Outbox persisten estado y eventos de forma atómica. El Publicador de Eventos envía mensajes a Azure Service Bus, mientras el Gestor de Estado y Conciliación controla resultados pendientes o inconsistentes. Los demás servicios se muestran en el C4 de Contenedores y no se descomponen individualmente para evitar redundancia; los mismos principios se aplican de manera consistente.

6. ARQUITECTURA DE CANALES Y EXPERIENCIA DIGITAL

6.1 Aplicación web SPA

La aplicación web se implementa como SPA y consume exclusivamente el BFF Web a través del API Gateway. No accede directamente a microservicios ni sistemas empresariales. Debe aplicar controles de seguridad del navegador, protección de contenido, manejo seguro de tokens, validación de entrada y monitoreo de experiencia.

Segunda justificación teórica: una SPA separa la experiencia de usuario de los servicios de negocio y permite evolucionar ambos ciclos de entrega mediante contratos estables. Además, reduce recargas completas y favorece una interacción fluida, siempre que el manejo de estado, accesibilidad y seguridad del navegador se mantenga controlado.

6.2 Aplicación móvil multiplataforma

Desde una mirada operativa, react Native se propone como opción principal para la aplicación móvil multiplataforma y Flutter se mantiene como alternativa. La selección final debe considerar experiencia del equipo, compatibilidad con SDK biométricos, estrategia de actualización, rendimiento, accesibilidad y soporte de seguridad del dispositivo.

Segunda justificación teórica: React Native permite compartir una parte relevante del código entre plataformas sin renunciar al acceso a capacidades nativas como biometría, almacenamiento seguro y notificaciones. Esta combinación reduce duplicidad de desarrollo y, al mismo tiempo, conserva la posibilidad de implementar módulos nativos cuando el rendimiento o un SDK especializado lo requiera.

6.3 BFF Web

El BFF Web adapta contratos y composición de respuestas para la SPA, reduce llamadas desde el navegador y encapsula particularidades de autenticación y presentación. En la operación diaria, no debe convertirse en un repositorio de reglas de negocio; estas permanecen en los servicios de dominio.

Segunda justificación teórica: un BFF específico para web evita que la SPA dependa de contratos genéricos o de la granularidad interna de los microservicios. También permite componer respuestas, reducir viajes de red y aplicar cambios de presentación sin trasladar lógica de canal a los dominios de negocio.

6.4 BFF Mobile

El BFF Mobile atiende necesidades específicas de conectividad, versiones de aplicación, payloads y experiencia móvil. Por eso, también facilita el onboarding y la integración con capacidades del dispositivo sin exponer directamente los servicios internos.

Segunda justificación teórica: el BFF Mobile permite adaptar tamaño de payload, compatibilidad de versiones y comportamiento ante conectividad variable, aspectos que no son equivalentes a los de un navegador. Al aislar estas necesidades, la evolución de la aplicación móvil no obliga a contaminar los servicios de negocio con condiciones propias del dispositivo.

6.5 API Gateway

El API Gateway centraliza publicación, versionado, validación de tokens, cuotas, rate limiting, políticas, transformación controlada y observabilidad de APIs. No debe contener lógica de negocio ni sustituir a los BFF o a los servicios de dominio.

Segunda justificación teórica: concentrar autenticación técnica, cuotas, versionado, rate limiting y políticas de exposición evita duplicar controles transversales en cada API. El Gateway también actúa como frontera estable frente a cambios internos, reduciendo el acoplamiento de los canales con la topología real de los servicios.

6.6 Gestión de sesiones y clientes frecuentes

En el escenario planteado, la sesión se mantiene conforme a las capacidades del proveedor de identidad y a políticas de seguridad de BP. Para clientes frecuentes se optimizan consultas aptas para caché y modelos de lectura, sin almacenar información sensible más tiempo del necesario ni relajar autenticación. La biometría local puede facilitar acceso seguro, pero no elimina controles de identidad ni autorización.

7. AUTENTICACIÓN, IDENTIDAD Y ONBOARDING

7.1 Arquitectura de identidad

La autenticación se delega al proveedor corporativo de identidad. Los canales y APIs confían en tokens emitidos por ese proveedor, validando firma, emisor, audiencia, expiración y permisos. Los servicios no administran contraseñas del cliente ni implementan mecanismos de autenticación paralelos.

Segunda justificación teórica: delegar la autenticación en un proveedor corporativo crea una única fuente de confianza y facilita aplicar políticas homogéneas de enrolamiento, revocación y recuperación de cuenta. Esto evita almacenar contraseñas en servicios de negocio y reduce la cantidad de componentes que procesan credenciales sensibles.

7.2 OAuth 2.0 y OpenID Connect

OAuth 2.0 se utiliza para autorización y OpenID Connect para autenticación e identidad. La decisión busca algo concreto: el Access Token autoriza el consumo de APIs y el ID Token representa el resultado de autenticación para el cliente. Los alcances y claims deben limitarse a lo estrictamente requerido.

Segunda justificación teórica: OAuth 2.0 permite autorización delegada mediante tokens limitados por alcance y tiempo, evitando compartir credenciales del usuario con las APIs. OpenID Connect añade una capa estandarizada de identidad sobre ese flujo, lo que facilita validar autenticación, sesión y claims de forma interoperable entre canales y servicios.

7.3 Authorization Code Flow con PKCE

Authorization Code Flow con PKCE es el flujo recomendado para SPA y aplicación móvil. Con ese criterio, el canal genera Code Verifier y Code Challenge, recibe un código de autorización y lo intercambia junto con el verificador. PKCE reduce el riesgo de interceptación del código en clientes públicos.

Segunda justificación teórica: PKCE vincula el código de autorización con un secreto efímero generado por el cliente, por lo que un código interceptado no puede canjearse por sí solo. Este principio es especialmente relevante en SPA y aplicaciones móviles, donde no es posible proteger de manera confiable un client secret permanente.

7.4 Autenticación multifactor

MFA se aplica conforme a política, perfil de riesgo, dispositivo, ubicación y naturaleza de la operación. Deben definirse métodos permitidos, recuperación de cuenta, enrolamiento, excepciones y protección contra fatiga de autenticación.

Segunda justificación teórica: MFA aplica defensa en profundidad al exigir evidencia adicional a la contraseña y reduce el impacto de credenciales filtradas, reutilizadas o obtenidas mediante phishing. También permite graduar el nivel de confianza de la sesión conforme al riesgo y a las políticas de BP.

7.5 Step-up authentication

Las operaciones sensibles pueden requerir una autenticación reforzada aun cuando exista una sesión válida. El step-up authentication debe activarse según reglas de monto, beneficiario nuevo, cambio de dispositivo, señales de fraude u otros criterios aprobados.

Segunda justificación teórica: el step-up authentication aplica autenticación adaptativa, elevando el nivel de aseguramiento solo cuando el contexto o la operación lo justifican. Así se protege con mayor rigor una transferencia sensible sin imponer la misma fricción en cada consulta de bajo riesgo.

7.6 Biometría local del dispositivo

La biometría local desbloquea credenciales o claves protegidas por el sistema operativo del dispositivo. La decisión busca algo concreto: la plantilla biométrica no se envía a BP. Este mecanismo mejora la experiencia, pero no sustituye al proveedor corporativo de identidad ni debe considerarse suficiente por sí solo para operaciones de alto riesgo.

Segunda justificación teórica: al ejecutarse localmente, la biometría aprovecha el hardware seguro del dispositivo y evita que BP reciba o almacene la plantilla biométrica. Su uso como mecanismo de desbloqueo mejora la experiencia sin convertirla en una identidad central ni alterar la validación realizada por el proveedor corporativo.

7.7 Passkeys y FIDO2

Passkeys y FIDO2 se plantean como evolución futura para reducir dependencia de contraseñas y resistencia al phishing. En consecuencia, su adopción requiere validar compatibilidad del proveedor de identidad, recuperación de cuenta, sincronización entre dispositivos, soporte al cliente y regulación.

Segunda justificación teórica: Passkeys/FIDO2 utiliza criptografía asimétrica vinculada al dominio legítimo, por lo que elimina el secreto compartido susceptible de phishing o reutilización. Se mantiene como evolución porque su adopción exige resolver compatibilidad, recuperación de cuenta y soporte operativo antes de convertirla en mecanismo principal.

7.8 Onboarding con reconocimiento facial

El onboarding móvil recopila datos e imágenes, ejecuta reconocimiento facial y prueba de vida mediante un proveedor especializado, valida identidad contra información oficial, crea o actualiza el perfil digital, registra al cliente en el proveedor de identidad y configura MFA. Los resultados deben conservar evidencia y trazabilidad de acuerdo con regulación y minimización de datos.

Diagrama de Secuencia - Onboarding y Autenticación

Sistema de Banca por Internet BP

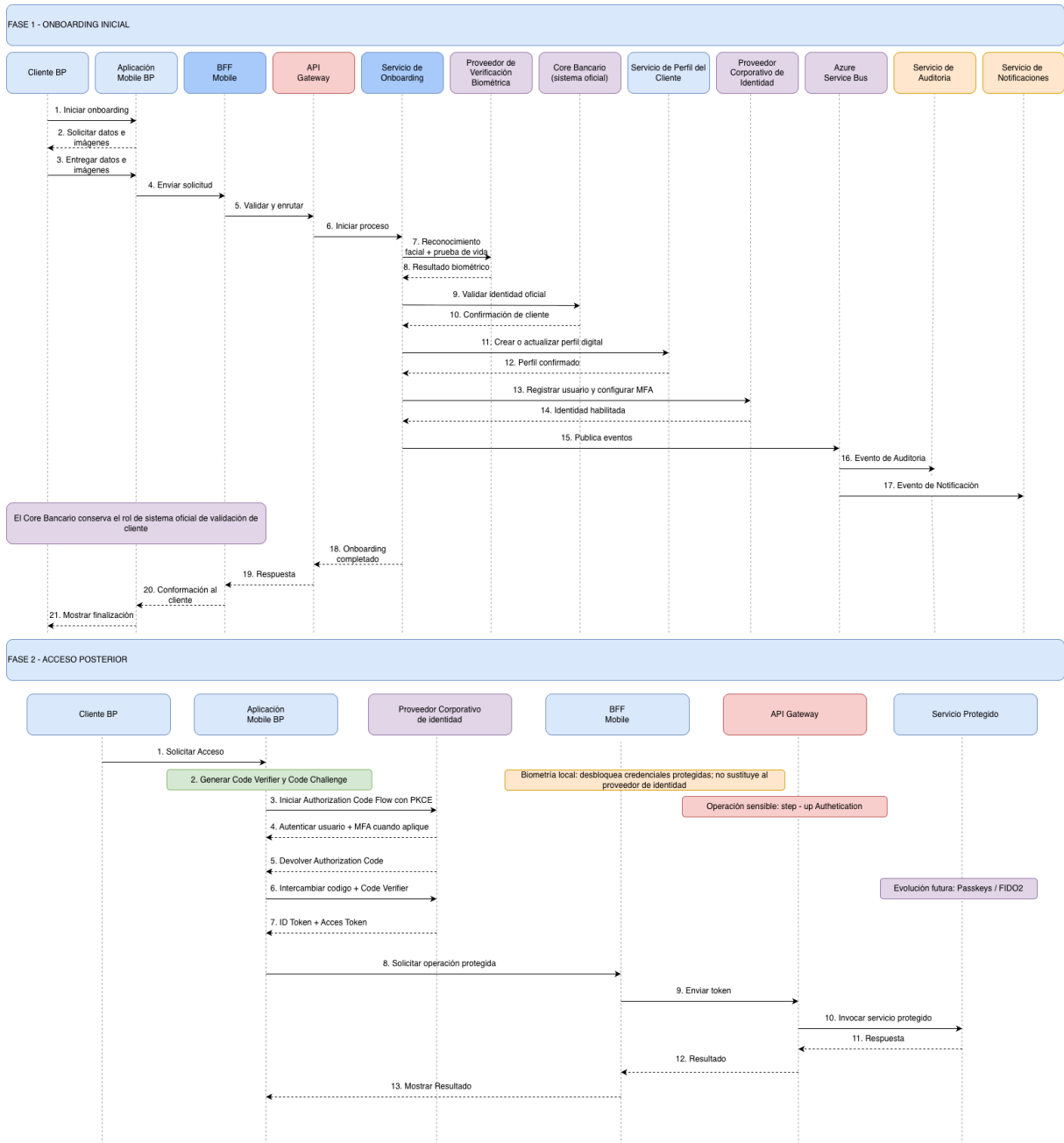


Figura 4. Secuencia de onboarding inicial y acceso posterior.

Segunda justificación teórica: un proveedor especializado concentra capacidades de prueba de vida, detección de suplantación y actualización de modelos que requieren conocimiento y evolución continua. Esta separación permite que BP gobierne el proceso y la evidencia sin asumir internamente toda la complejidad técnica y operativa del reconocimiento biométrico.

7.9 Gestión de secretos e identidades administradas

Azure Key Vault centraliza secretos, certificados y claves. Managed Identities permite que componentes Azure accedan a recursos sin almacenar credenciales estáticas. El acceso se concede por mínimo privilegio, se audita y se rota conforme a políticas institucionales.

Segunda justificación teórica: Azure Key Vault centraliza ciclo de vida, rotación y auditoría de secretos, evitando su dispersión en código, archivos o pipelines. Managed Identities elimina credenciales estáticas entre servicios y permite autorizar accesos mediante identidad y mínimo privilegio, reduciendo el riesgo de filtración y de rotaciones manuales.

8. SERVICIOS DE NEGOCIO

8.1 Perfil del cliente

Consolida y expone datos digitales del cliente. Consulta el sistema complementario y el Core mediante la Capa Anticorrupción, aplica reglas de privacidad y evita duplicar información oficial innecesariamente.

Segunda justificación teórica: separar Perfil del cliente concentra reglas de privacidad, composición y precedencia de datos en un único límite de dominio. Esto evita que cada canal integre directamente sistemas oficiales y reduce respuestas inconsistentes ante cambios en las fuentes de información.

8.2 Productos

De forma concreta, presenta cuentas y productos disponibles para el cliente. Utilizando información oficial del Core y modelos de lectura optimizados cuando sea seguro y consistente.

Segunda justificación teórica: un servicio de Productos desacopla la presentación digital de la estructura interna del Core y permite construir modelos de lectura adecuados para los canales. También concentra reglas de visibilidad y elegibilidad sin duplicarlas en Web, Mobile o BFF.

8.3 Movimientos

Consulta el histórico de movimientos, aplica paginación y filtros. A partir de ahí, y puede utilizar caché de corta duración para consultas repetitivas que no comprometan exactitud ni privacidad.

Segunda justificación teórica: separar Movimientos permite optimizar paginación, filtros y modelos de consulta sin afectar las operaciones transaccionales. Además, mantiene aisladas las políticas de frescura y caché de un caso de uso predominantemente de lectura.

8.4 Transferencias

Gestiona transferencias propias e interbancarias, idempotencia, Saga, estados, integración, publicación de eventos y conciliación. Con ese criterio, no reemplaza el registro financiero del Core.

Segunda justificación teórica: Transferencias requiere un límite propio porque combina reglas, estados, idempotencia, coordinación y conciliación con una criticidad superior a una consulta. Su aislamiento permite aplicar controles y escalabilidad específicos sin extender esa complejidad a otros dominios.

8.5 Pagos

Orquesta validaciones, ejecución y seguimiento de pagos. Debe aplicar idempotencia, límites, antifraude, auditoría y conciliación según el tipo de pago.

Segunda justificación teórica: Pagos se mantiene como capacidad independiente porque puede integrar receptores, reglas y estados distintos de una transferencia. El límite facilita incorporar nuevos tipos de pago y sus conciliaciones sin modificar el ciclo de vida del Servicio de Transferencias.

8.6 Beneficiarios

Aquí conviene precisar que administra beneficiarios como dato propio del dominio digital. Con validaciones, autorización reforzada y trazabilidad de altas, modificaciones y eliminaciones.

Segunda justificación teórica: Beneficiarios posee datos y reglas de ciclo de vida propias, incluidos alta, modificación, validación y autorización reforzada. Separarlo reduce el acoplamiento con transferencias y permite reutilizar la capacidad desde pagos u otros productos digitales.

8.7 Límites y reglas transaccionales

Evalúa montos, frecuencia, tipo de operación, perfil del cliente y políticas institucionales. La decisión busca algo concreto: sus decisiones deben ser trazables y versionadas.

Segunda justificación teórica: centralizar límites y reglas evita interpretaciones distintas entre pagos, transferencias y canales. Además, permite versionar políticas y explicar qué conjunto de reglas estuvo vigente cuando se tomó una decisión transaccional.

8.8 Antifraude

Consume señales de sesión, dispositivo y transacción para determinar riesgo, bloquear, desafiar o permitir operaciones. En consecuencia, la integración debe soportar timeouts y decisiones conservadoras ante indisponibilidad según política.

Segunda justificación teórica: el antifraude necesita evolucionar y escalar de manera independiente porque combina señales, modelos y reglas que cambian con el comportamiento de ataque. Su separación también permite aplicar decisiones homogéneas a varios procesos sin incorporar lógica de riesgo dentro de cada servicio de negocio.

8.9 Onboarding

Coordina captura, verificación biométrica, validación con sistemas oficiales, creación del perfil digital, registro de identidad y notificación. Ese punto debe quedar visible desde el diseño.

Segunda justificación teórica: Onboarding coordina un proceso de larga duración con pasos, evidencia y proveedores que no pertenecen al acceso cotidiano. Mantenerlo separado limita el tratamiento de datos sensibles y permite modificar validaciones de alta sin afectar autenticación o perfil.

8.10 Notificaciones

Consume eventos y orquesta push, correo y SMS. Mantiene preferencias, plantillas, reintentos, estado de entrega e idempotencia del consumidor.

Segunda justificación teórica: Notificaciones se desacopla del proceso financiero para que latencia o indisponibilidad de push, correo o SMS no bloquee la operación principal. El componente también permite escalar, reintentar y gobernar plantillas y preferencias de forma independiente.

8.11 Auditoría

Registra acciones relevantes del cliente y decisiones críticas con controles de integridad e inmutabilidad. La decisión busca algo concreto: es independiente de los logs técnicos.

Segunda justificación teórica: Auditoría se mantiene como capacidad separada para aplicar retención, integridad y controles de acceso diferentes a los de los logs operativos. Esta separación protege el valor probatorio de los registros y evita que cambios de observabilidad alteren la evidencia funcional.

8.12 Conciliación

Identifica operaciones pendientes o inconsistentes entre servicios digitales. A partir de ahí, core y red interbancaria, y activa resolución automática o manual conforme a reglas.

Segunda justificación teórica: en sistemas distribuidos con consistencia eventual siempre pueden existir respuestas ambiguas o estados divergentes; la conciliación proporciona un mecanismo explícito de convergencia. Al aislarla, se pueden ejecutar comparaciones, reintentos y resolución manual sin bloquear el flujo en línea.

9. INTEGRACIÓN CON SISTEMAS EMPRESARIALES

9.1 Integración con el Core bancario

Toda operación financiera oficial se ejecuta o confirma en el Core bancario. En la operación diaria, los servicios digitales no mantienen saldos oficiales ni sustituyen los asientos financieros. La integración debe utilizar contratos controlados, autenticación mutua, timeouts, trazabilidad y manejo explícito de respuestas ambiguas.

9.2 Integración con el sistema complementario del cliente

El sistema complementario aporta información detallada del cliente que no se encuentre disponible en el Core. Por eso, la solución debe definir precedencia, frescura, privacidad y manejo de discrepancias para evitar mostrar datos contradictorios.

9.3 Anti-Corruption Layer

La Capa Anticorrupción traduce protocolos, formatos, códigos y semántica de los sistemas empresariales hacia contratos del dominio digital. Así evita que particularidades heredadas se propaguen a microservicios y canales. También concentra adaptación de errores, versionado y políticas de resiliencia de integración.

Segunda justificación teórica: la Capa Anticorrupción preserva el modelo del dominio digital al traducir conceptos y errores de sistemas heredados, evitando que sus particularidades se conviertan en dependencias permanentes. También confina los cambios de contratos externos a una frontera controlada y reduce el impacto sobre canales y microservicios.

9.4 Integración mediante API Gateway

El criterio adoptado es el siguiente: el API Gateway gobierna APIs expuestas a canales y consumidores autorizados. Las integraciones internas con Core y sistemas complementarios se realizan a través de adaptadores y la Capa Anticorrupción; el API Gateway no debe convertirse en un bus de integración general ni contener lógica de negocio.

Segunda justificación teórica: utilizar el API Gateway como frontera de exposición mantiene separadas las políticas de consumo externo de la integración interna con sistemas empresariales. Esta división evita convertir el Gateway en un componente de lógica o transformación profunda y conserva responsabilidades operativas claras.

9.5 Comunicación REST

REST se utiliza para consultas y comandos que requieren respuesta inmediata. En la operación diaria, los contratos deben ser versionados, idempotentes cuando aplique, paginados para colecciones y consistentes en códigos de error. Se recomienda evitar cadenas síncronas profundas que amplifiquen latencia y fallas.

Segunda justificación teórica: REST aprovecha una interfaz uniforme, comunicación stateless y semántica HTTP conocida, lo que mejora interoperabilidad y observabilidad. Su adopción también facilita versionado y pruebas de contrato, siempre que se eviten cadenas síncronas extensas y se utilice mensajería cuando no se requiere respuesta inmediata.

9.6 Mensajería con Azure Service Bus

Azure Service Bus soporta eventos de auditoría, notificación y procesos posteriores, además de comandos asíncronos cuando se requiera entrega confiable. Por eso, las suscripciones independientes permiten que cada consumidor procese eventos sin bloquear a los demás. Deben configurarse reintentos, dead-letter queue y observabilidad.

Segunda justificación teórica: Azure Service Bus introduce desacoplamiento temporal y entrega durable, de modo que productor y consumidor no necesitan estar disponibles al mismo tiempo. Sus colas, suscripciones y dead-letter queue soportan reintentos controlados, aislamiento de consumidores y recuperación de mensajes fallidos.

9.7 Gestión de errores de integración

Los errores se clasifican en funcionales, transitorios, técnicos y ambiguos. Los funcionales se devuelven sin reintento; los transitorios pueden reintentarse con límite y backoff; los ambiguos requieren consulta de estado o conciliación antes de repetir una operación financiera.

9.8 Timeouts, reintentos y circuit breakers

En este punto, cada dependencia debe tener timeout explícito. Los reintentos se limitan a operaciones seguras o idempotentes y deben incluir backoff y jitter. El circuit breaker evita saturar dependencias fallidas y permite degradación controlada. Sus umbrales deben ajustarse con métricas reales.

Segunda justificación teórica: los timeouts acotan el consumo de recursos ante dependencias lentas, los reintentos recuperan fallas transitorias y el circuit breaker evita insistir sobre un servicio degradado. En conjunto, estos patrones reducen fallas en cascada y mantienen predecible el comportamiento de la plataforma bajo presión.

10. PROCESAMIENTO TRANSACCIONAL

10.1 Transferencias entre cuentas propias

El flujo valida token, cuentas, límites, riesgo e Idempotency Key. El Servicio de Transferencias inicia la Saga, solicita la operación al Core mediante la Capa Anticorrupción, persiste el estado y el Transactional Outbox de forma atómica, responde al cliente y publica eventos de manera asíncrona.

10.2 Transferencias interbancarias

Las transferencias interbancarias incorporan la red o plataforma externa y estados potencialmente más prolongados. La decisión busca algo concreto: el servicio debe distinguir aceptación, procesamiento, confirmación, rechazo y estado desconocido. La conciliación es obligatoria cuando no exista confirmación concluyente.

10.3 Pagos

Los pagos siguen principios equivalentes: validación, idempotencia, límites, antifraude, ejecución oficial, persistencia de estado, publicación de eventos y conciliación. A partir de ahí, las reglas específicas dependen del tipo de pago y del sistema receptor.

10.4 Saga orquestada

La Saga orquestada controla pasos, estados y compensaciones de procesos distribuidos. El orquestador no sustituye transacciones locales ni garantiza atomicidad global; hace explícita la coordinación y permite recuperar o compensar fallas parciales.

Segunda justificación teórica: Saga permite coordinar transacciones locales sin depender de un commit distribuido, que resulta frágil y poco compatible con servicios autónomos. La variante orquestada hace visibles el estado, las decisiones y las compensaciones, facilitando recuperación, auditoría y operación de un flujo financiero complejo.

10.5 Idempotencia

Cada solicitud transaccional incluye una Idempotency Key asociada al cliente, operación y ventana de validez. El Gestor de Idempotencia registra el resultado y devuelve la respuesta previa ante reintentos equivalentes. Solicitudes con la misma clave y contenido diferente deben rechazarse.

Segunda justificación teórica: en redes y mensajería no puede asumirse entrega exactamente una vez; los clientes y componentes pueden repetir solicitudes por timeout o pérdida de respuesta. La idempotencia convierte esos reintentos en operaciones seguras y evita débitos, pagos o registros duplicados.

10.6 Transactional Outbox

El estado de la operación y el evento pendiente se guardan en una misma transacción local. La decisión busca algo concreto: un publicador independiente lee el Transactional Outbox y envía el evento a Azure Service Bus. Los consumidores siguen siendo idempotentes porque la entrega puede repetirse.

Segunda justificación teórica: Transactional Outbox elimina el problema de doble escritura entre base de datos y mensajería al registrar ambos resultados dentro de una misma transacción local. De esta forma, una caída entre persistencia y publicación no pierde el evento y el publicador puede reanudar el envío posteriormente.

10.7 Conciliación

La conciliación compara estados digitales con Core y red interbancaria, identifica diferencias y determina actualización, reintento seguro, compensación o intervención manual. Con ese criterio, toda resolución debe quedar auditada y preservar evidencia.

Segunda justificación teórica: la conciliación actúa como control compensatorio frente a estados desconocidos, demoras externas y consistencia eventual. Su existencia garantiza que una operación no quede indefinidamente ambigua y que cualquier ajuste posterior preserve evidencia y trazabilidad.

10.8 Consistencia y manejo de transacciones distribuidas

Se adopta consistencia fuerte dentro de cada transacción local y consistencia eventual entre servicios. No se utiliza una transacción distribuida global. La arquitectura gestiona incertidumbre mediante estados explícitos, idempotencia, Saga, Transactional Outbox y conciliación.

Diagrama de Secuencia - Onboarding y Autenticación

Sistema de Banca por Internet BP

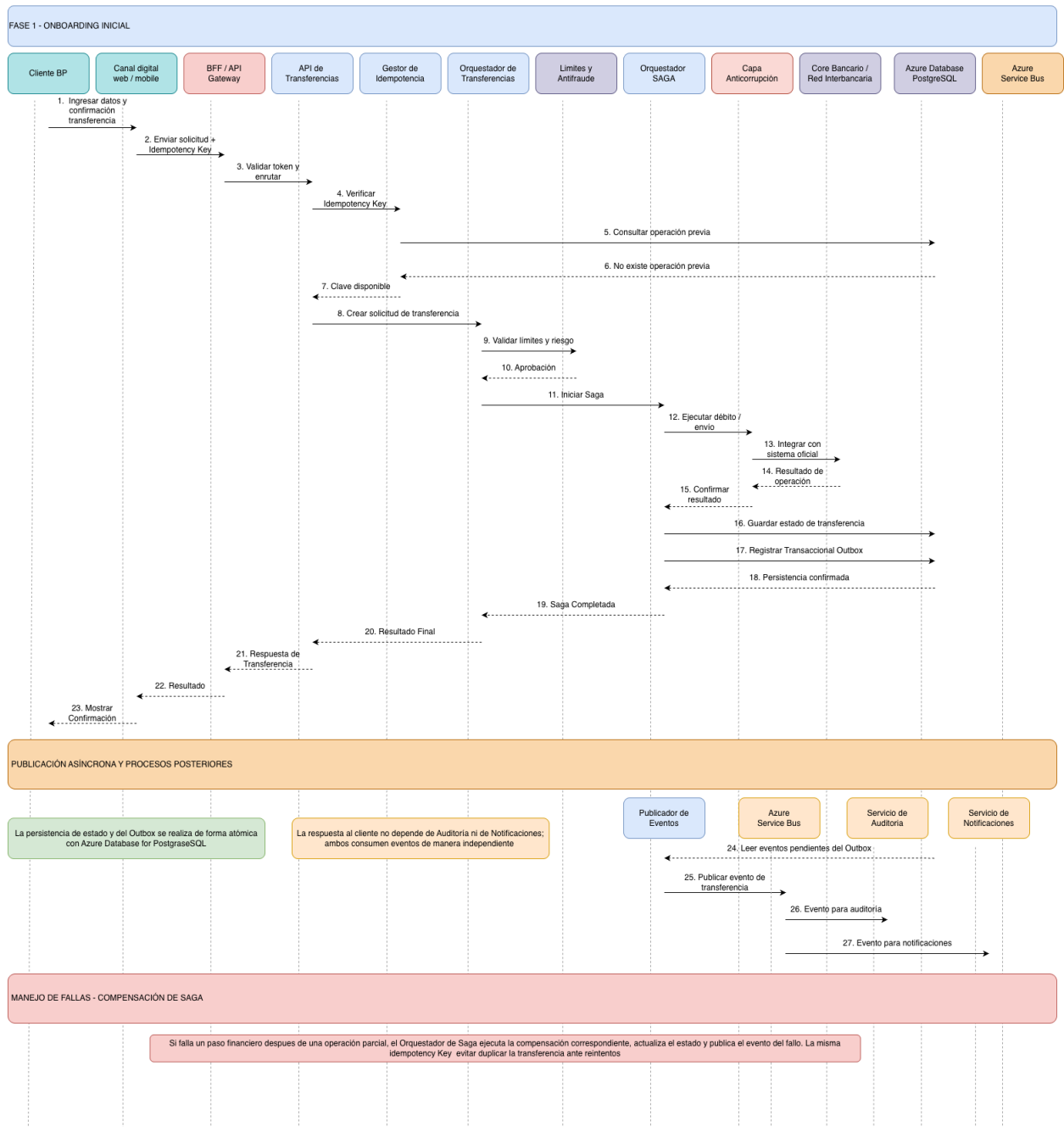


Figura 5. Secuencia de transferencia con idempotencia, Saga y publicación asíncrona.

Segunda justificación teórica: combinar consistencia fuerte local con consistencia eventual entre servicios respeta la autonomía de cada dominio y evita bloquear recursos a través de múltiples sistemas. Los estados explícitos y mecanismos de convergencia hacen visible la incertidumbre en lugar de ocultarla bajo una supuesta atomicidad global.

11. ARQUITECTURA DE DATOS

11.1 Principios de persistencia

Los datos se almacenan según propiedad, finalidad, sensibilidad y ciclo de vida. En consecuencia, se evita duplicar información oficial del Core y se mantiene trazabilidad de cualquier copia necesaria. Los esquemas y contratos evolucionan de forma compatible y con migraciones controladas.

11.2 Base de datos por servicio

Cada servicio dispone de base de datos o esquema lógico bajo su control. Otros servicios acceden mediante APIs o eventos, no mediante consultas directas. El aislamiento físico puede variar según criticidad, volumen, seguridad y costo; no todos los servicios requieren una instancia independiente.

Segunda justificación teórica: una base de datos o esquema bajo control de cada servicio refuerza encapsulamiento y evita acoplamiento mediante tablas compartidas. Esto permite aplicar migraciones, políticas de acceso y escalabilidad según las necesidades del dominio sin coordinar cada cambio con toda la plataforma.

11.3 Azure Database for PostgreSQL

Azure Database for PostgreSQL se utiliza para persistencia relacional administrada de dominios digitales. Puede almacenar beneficiarios, datos de onboarding, claves de idempotencia, estados de transferencias, Transactional Outbox, conciliación y controles propios. No se presenta como sistema oficial de registro financiero.

Segunda justificación teórica: PostgreSQL ofrece transacciones ACID, restricciones e integridad referencial apropiadas para estados operativos y reglas digitales que no deben quedar inconsistentes. Al utilizarlo como servicio administrado se incorporan capacidades de respaldo, alta disponibilidad y mantenimiento sin convertir al equipo en operador del motor, manteniendo al Core como registro financiero oficial.

11.4 Azure Cache for Redis

Azure Cache for Redis se utiliza como capa de optimización para consultas frecuentes, sesiones técnicas limitadas, rate limiting o datos efímeros apropiados. La decisión busca: la pérdida de caché no debe implicar pérdida del registro oficial ni corrupción funcional.

Segunda justificación teórica: Redis proporciona acceso de baja latencia a datos efímeros y reduce presión sobre servicios y bases de datos en lecturas repetitivas. Su uso como caché, y no como registro oficial, preserva la capacidad de reconstruir el estado cuando una entrada expira o el servicio se reinicia.

11.5 Patrón Cache-Aside

Con Cache-Aside, el servicio consulta primero la caché; ante ausencia obtiene el dato desde la fuente autorizada y lo almacena con expiración. A partir de ahí, las políticas de invalidación, TTL, clasificación y tamaño deben diseñarse por tipo de dato. No se cachean indiscriminadamente saldos u operaciones sensibles.

Segunda justificación teórica: Cache-Aside deja al servicio de dominio controlar cuándo leer, poblar e invalidar la caché, lo que permite ajustar frescura según el tipo de dato. También mantiene una ruta funcional hacia la fuente autorizada cuando la caché no está disponible, evitando convertirla en una dependencia obligatoria.

11.6 CQRS ligero

CQRS ligero separa modelos de lectura y escritura cuando la diferencia de volumen, forma o rendimiento lo justifica. No implica Event Sourcing ni duplicación general de toda la plataforma. Se aplica de forma selectiva a consultas de movimientos, productos o vistas consolidadas.

Segunda justificación teórica: separar modelos de lectura y escritura permite optimizar consultas sin comprometer las reglas e integridad del modelo transaccional. Mantenerlo ligero evita la complejidad de Event Sourcing y aplica la separación solo donde el volumen o la forma de consulta justifican el costo adicional.

11.7 Optimización para clientes frecuentes

La optimización combina caché segura, paginación, precálculo de vistas, compresión y respuestas adaptadas al canal. La frecuencia de uso no autoriza a conservar datos más allá de la política ni a omitir validaciones de seguridad.

11.8 Protección y clasificación de la información

La información debe clasificarse por sensibilidad y finalidad. La decisión: se aplican cifrado, enmascaramiento, mínimo privilegio, segregación, auditoría y controles de exportación. Los datos biométricos y de identidad requieren tratamiento reforzado y minimización.

11.9 Retención y ciclo de vida de los datos

Cada categoría de datos debe contar con período de retención, criterio de archivo, eliminación segura y responsable. Con ese criterio, la definición final corresponde a Cumplimiento, Legal, Riesgos y propietarios de información, considerando regulación y necesidades probatorias.

12. AUDITORÍA Y NOTIFICACIONES

12.1 Auditoría de acciones del cliente

Se auditan autenticaciones, altas y cambios de beneficiarios, consultas sensibles, transferencias, pagos, cambios de datos, decisiones de riesgo y acciones administrativas relevantes. En la operación diaria, cada registro incluye identidad, acción, fecha, resultado, origen, Correlation ID y referencia de negocio, evitando almacenar secretos o datos completos innecesarios.

12.2 Separación entre auditoría y logs operativos

La auditoría constituye evidencia funcional y de seguridad con retención y controles propios. Por eso, los logs operativos sirven para diagnóstico técnico y pueden tener ciclo de vida diferente. Un log no reemplaza un registro de auditoría, y la auditoría no debe llenarse con ruido técnico.

Segunda justificación teórica: auditoría y logs responden a finalidades, retenciones y audiencias distintas; mezclarlos dificulta preservar evidencia y aumenta el ruido operativo. La separación permite aplicar inmutabilidad y acceso restringido a los registros probatorios, mientras los logs se optimizan para diagnóstico y monitoreo.

12.3 Inmutabilidad e integridad de registros

El almacén de auditoría debe aplicar acceso restringido, escritura controlada, retención, sellado o mecanismos equivalentes de integridad y trazabilidad de cambios. La implementación concreta se validará con Seguridad y Cumplimiento según requisitos probatorios.

Segunda justificación teórica: los controles de inmutabilidad permiten detectar alteraciones y sostener confianza en la evidencia después de un incidente o disputa. También reducen el riesgo de que una identidad operativa con privilegios pueda modificar silenciosamente la historia de acciones registradas.

12.4 Eventos de auditoría

Desde una mirada operativa, los microservicios publican eventos de auditoría mediante Transactional Outbox y Azure Service Bus. El Canal de Auditoría se procesa de forma independiente. El consumidor aplica idempotencia para evitar registros duplicados y deriva eventos inválidos a dead-letter queue.

Segunda justificación teórica: publicar auditoría mediante Outbox y mensajería evita perder evidencia cuando el servicio confirma una operación y falla antes de enviar el evento. El consumo independiente permite aplicar idempotencia, retención y escalabilidad sin aumentar la latencia del flujo principal.

12.5 Notificaciones asíncronas

Las notificaciones se originan a partir de eventos de negocio confirmados. En la operación diaria, el Canal de Notificaciones es independiente del Canal de Auditoría. La falla de una entrega no revierte una operación financiera; se registra, reintenta y escala conforme a política.

Segunda justificación teórica: el procesamiento asíncrono desacopla la confirmación financiera de proveedores con latencia y disponibilidad variables. Esto permite responder al cliente con el estado real de la operación y gestionar entregas, reintentos o canales alternativos sin revertir una transacción confirmada.

12.6 Notificaciones push

Push es el mecanismo preferido para avisos oportunos cuando el cliente tiene una aplicación registrada. Por eso, debe evitar incluir información sensible en la pantalla bloqueada y gestionar tokens de dispositivo inválidos o expirados.

12.7 Correo electrónico

El correo permite confirmaciones y comunicaciones con mayor detalle. Las plantillas deben estar versionadas, evitar datos sensibles y cumplir políticas de marca, privacidad y prevención de fraude.

12.8 SMS

En el escenario planteado, SMS funciona como canal alternativo o complementario, sujeto a costo, cobertura y riesgo de suplantación. El contenido debe ser breve, no incluir enlaces inseguros ni información financiera detallada.

12.9 Reintentos y manejo de entregas fallidas

Los reintentos utilizan backoff y límites por canal. En la operación diaria, la dead-letter queue conserva mensajes que exceden el umbral para análisis y reproceso controlado. El servicio mantiene idempotencia del consumidor y estado de entrega por mensaje y destinatario.

Diagrama de Auditoría y Notificaciones

Sistema de Banca por Internet BP

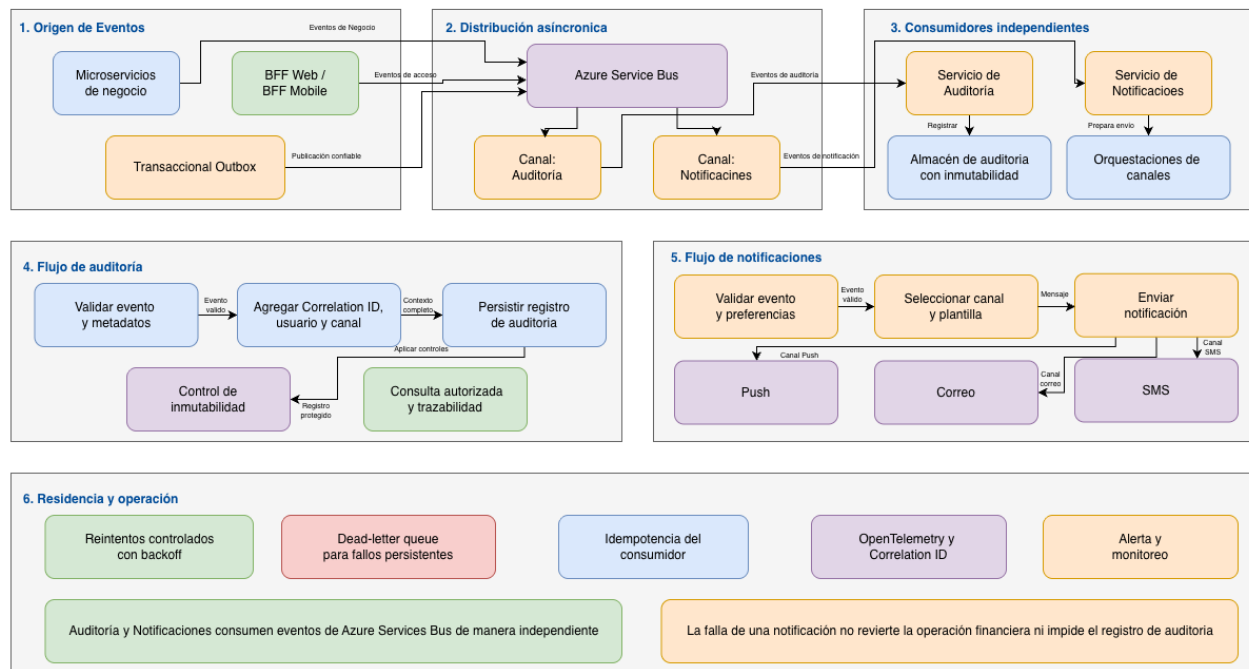


Figura 6. Arquitectura de auditoría y notificaciones por canales independientes.

Segunda justificación teórica: la dead-letter queue conserva mensajes que no pudieron procesarse y evita ciclos infinitos de reintento que degradan al sistema. Mantener estado e idempotencia por destinatario permite reprocesar de forma controlada sin enviar notificaciones duplicadas.

13. SEGURIDAD

13.1 Modelo de seguridad por capas

En términos de diseño, la seguridad combina controles preventivos, detectivos y de respuesta en perímetro, identidad, aplicación, integración, datos, plataforma y operación. Ningún control se considera suficiente de forma aislada.

Segunda justificación teórica: la defensa en profundidad asume que cualquier control puede fallar y superpone barreras preventivas, detectivas y de respuesta. Así, una vulnerabilidad en el canal no implica automáticamente acceso a datos o ejecución transaccional, porque identidad, red, aplicación y datos mantienen controles independientes.

13.2 Protección perimetral

Azure Front Door, WAF, protección DDoS, TLS, restricciones de origen y monitoreo protegen el ingreso. En la operación diaria, la configuración debe seguir políticas corporativas y exposición mínima.

Segunda justificación teórica: Azure Front Door proporciona un punto global y controlado de entrada, reduce exposición directa de los backends y facilita aplicar enrutamiento y protección consistentes. Esta frontera también permite absorber cambios de topología interna sin alterar los endpoints utilizados por los clientes.

13.3 Web Application Firewall

El WAF bloquea patrones comunes de ataque y aplica reglas administradas y específicas. Por eso, las reglas deben probarse para evitar falsos positivos y mantenerse actualizadas.

Segunda justificación teórica: el WAF inspecciona tráfico HTTP en la capa de aplicación y bloquea patrones conocidos antes de que alcancen los servicios. Centralizar reglas permite actualizar protección de forma homogénea y registrar intentos de ataque sin depender de que cada aplicación implemente sus propios filtros.

13.4 Protección DDoS

La protección DDoS reduce impacto de ataques volumétricos. Debe complementarse con capacidad, límites, runbooks, alertas y coordinación operativa.

Segunda justificación teórica: la protección DDoS responde a ataques volumétricos que no pueden mitigarse únicamente desde la aplicación y protege la disponibilidad de la plataforma. Su integración con telemetría y runbooks facilita detectar, escalar y coordinar la respuesta ante saturación maliciosa.

13.5 Gestión de identidades y accesos

El criterio adoptado es el siguiente: se aplica mínimo privilegio, segregación de funciones, revisiones periódicas. MFA para acceso administrativo y cuentas de emergencia controladas.

Segunda justificación teórica: mínimo privilegio y segregación de funciones reducen tanto abuso deliberado como errores accidentales al limitar qué puede ejecutar cada identidad. Las revisiones periódicas y cuentas de emergencia controladas aseguran que el acceso administrativo tenga vigencia, propósito y evidencia.

13.6 Protección de APIs

Las APIs validan tokens, scopes, audiencia y permisos. Por esa razón, aplican rate limiting, validación de entrada, versionado, protección contra replay e idempotencia en comandos críticos.

Segunda justificación teórica: validar token, audiencia, scopes y permisos en cada frontera evita que un token legítimo sea utilizado fuera de su propósito. Rate limiting, protección contra replay e idempotencia limitan abuso automatizado y reducen el impacto de solicitudes repetidas sobre operaciones críticas.

13.7 Cifrado en tránsito y en reposo

Todas las comunicaciones utilizan TLS aprobado. Por eso, los datos se cifran en reposo mediante capacidades administradas y, cuando el riesgo lo requiera, con claves controladas por BP.

Segunda justificación teórica: el cifrado en tránsito protege confidencialidad e integridad frente a interceptación o manipulación, mientras el cifrado en reposo limita la exposición ante acceso físico o copia no autorizada. Mantener ambos controles evita depender de una sola frontera para proteger información sensible.

13.8 Azure Key Vault

Azure Key Vault centraliza secretos, certificados y claves, con acceso auditado, rotación y separación por ambiente.

Segunda justificación teórica: Key Vault permite separar secretos del ciclo de vida del código y aplicar rotación sin reconstruir artefactos. Sus controles de acceso y auditoría ofrecen evidencia de uso y reducen la dispersión de material criptográfico entre aplicaciones y ambientes.

13.9 Managed Identities

En este punto, managed Identities reemplaza credenciales embebidas para accesos entre servicios Azure. Reduciendo riesgo de filtración y carga de rotación.

Segunda justificación teórica: Managed Identities sustituye secretos permanentes por una identidad administrada cuyo token se obtiene dinámicamente. Esto elimina credenciales embebidas y permite revocar o limitar accesos mediante roles sin distribuir nuevas claves.

13.10 Seguridad de contenedores

Se aplican imágenes mínimas, escaneo de vulnerabilidades. Esto permite que firma o procedencia confiable, ejecución sin privilegios, políticas de red, parcheo y control de admisión.

Segunda justificación teórica: imágenes mínimas y verificadas reducen superficie de ataque y hacen reproducible el artefacto desplegado. La ejecución sin privilegios, las políticas de red y el control de admisión limitan movimiento lateral y evitan que una carga comprometida obtenga capacidades innecesarias.

13.11 Seguridad de datos

En términos de diseño, se aplican clasificación, minimización, enmascaramiento, control de acceso, cifrado, monitoreo y borrado seguro según ciclo de vida.

13.12 Prevención y detección de fraude

El motor antifraude evalúa señales transaccionales y contextuales. Las reglas deben ser gobernadas, versionadas y auditables, con revisión de falsos positivos.

Segunda justificación teórica: combinar señales contextuales y transaccionales permite evaluar riesgo más allá de la autenticación inicial, que por sí sola no prueba que cada operación sea legítima. El gobierno y versionado de reglas conserva explicabilidad y permite revisar falsos positivos sin perder trazabilidad.

13.13 Trazabilidad y no repudio

Desde una mirada operativa, la combinación de identidad fuerte, step-up authentication. Registros de auditoría, Correlation ID, evidencia de decisiones y sellado de integridad aporta trazabilidad y soporte al no repudio.

Segunda justificación teórica: identidad fuerte, autenticación reforzada y evidencia íntegra vinculan una acción con su actor, contexto y resultado. Esta combinación dificulta negar posteriormente una operación y permite reconstruirla sin depender de una única fuente de registro.

14. DISPONIBILIDAD, RESILIENCIA Y RECUPERACIÓN

14.1 Alta disponibilidad multizona

Los componentes críticos de la región primaria se distribuyen entre al menos dos zonas de disponibilidad. Las réplicas deben evitar dependencia de una zona y conservar estado en servicios administrados con alta disponibilidad.

Segunda justificación teórica: distribuir réplicas entre zonas separa dominios de falla y evita que un incidente de infraestructura local elimine toda la capacidad del servicio. El diseño también permite mantenimiento y reemplazo de instancias sin interrumpir completamente la atención.

14.2 Tolerancia a fallos

La solución limita el efecto de fallas mediante aislamiento, timeouts, reintentos controlados, circuit breakers, colas, idempotencia y degradación. La decisión busca: cada dependencia debe tener comportamiento definido ante indisponibilidad.

Segunda justificación teórica: aislar dependencias y definir comportamientos degradados evita que una falla local se transforme en indisponibilidad general. Los límites de espera y de reintento protegen hilos, conexiones y colas, manteniendo recursos disponibles para solicitudes que todavía pueden completarse.

14.3 Auto-healing

AKS reinicia cargas no saludables y redistribuye réplicas. En consecuencia, las sondas deben distinguir inicio, disponibilidad y vida. El auto-healing no reemplaza análisis de causa ni puede ocultar fallas repetitivas.

Segunda justificación teórica: el auto-healing compara el estado real con el estado deseado y reemplaza automáticamente instancias no saludables, reduciendo el tiempo medio de recuperación. Esta automatización es adecuada para fallas repetibles de infraestructura, mientras los incidentes persistentes continúan requiriendo análisis de causa.

14.4 Escalabilidad automática

Los componentes sin estado escalan horizontalmente con métricas de CPU, memoria, latencia, concurrencia o profundidad de cola. Los límites deben proteger dependencias como Core y red interbancaria.

Segunda justificación teórica: el escalado automático ajusta capacidad según señales observables y evita depender de intervención manual durante picos. También mejora eficiencia de costo al reducir recursos cuando la demanda baja, siempre que se establezcan límites para proteger al Core y demás dependencias.

14.5 Región secundaria warm standby

La región secundaria mantiene capacidad mínima preparada, configuración reproducible y componentes esenciales listos para ampliarse. No se plantea operación activo-activo regional como decisión inicial.

Segunda justificación teórica: warm standby equilibra continuidad y costo al mantener componentes y configuración preparados sin duplicar toda la capacidad productiva. Frente a activo-activo, reduce complejidad de consistencia regional y permite alcanzar un RTO razonable mediante ampliación y conmutación controlada.

14.6 Recuperación ante desastres

La recuperación incluye detección, decisión, conmutación, ampliación de capacidad, verificación de datos, reapertura controlada y retorno posterior. La decisión busca algo concreto: los procedimientos deben estar documentados y ensayados.

14.7 RTO y RPO

Se asumen inicialmente RTO de 60 minutos y RPO de 15 minutos. A partir de ahí, estos valores deben validarse por capacidad de negocio, dependencia, regulación y costo; podrían diferir entre consultas, transferencias y servicios auxiliares.

Segunda justificación teórica: definir RTO y RPO convierte la continuidad en objetivos medibles y permite relacionar inversión, arquitectura y criticidad del negocio. Estos valores orientan frecuencia de replicación, capacidad reservada y procedimientos de recuperación, y deben comprobarse mediante ejercicios reales.

14.8 Estrategia de respaldos

Se requieren respaldos automáticos, retención aprobada, cifrado, separación de permisos y pruebas de restauración. Un respaldo no se considera válido hasta comprobar que puede recuperarse.

Segunda justificación teórica: los respaldos protegen frente a corrupción lógica, eliminación accidental o ataques que pueden replicarse también a sistemas de alta disponibilidad. La prueba de restauración valida que la copia es utilizable y que tiempos, permisos y dependencias cumplen el objetivo de recuperación.

14.9 Pruebas de recuperación

En la práctica, se realizan ejercicios periódicos de restauración y conmutación. Con medición real de RTO/RPO, hallazgos, responsables y acciones correctivas.

14.10 Degradación controlada del servicio

Ante fallas parciales, la plataforma puede mantener consultas disponibles, suspender temporalmente operaciones sensibles o informar estados pendientes. La decisión busca: nunca debe presentar como exitosa una operación no confirmada.

15. ARQUITECTURA DE DESPLIEGUE EN AZURE

15.1 Vista general de despliegue

El criterio adoptado es el siguiente: los clientes acceden mediante Azure Front Door y WAF. El API Gateway enruta hacia BFF y microservicios en AKS. Azure Service Bus desacopla procesos; Azure Functions ejecuta tareas apropiadas; Azure Key Vault protege secretos; Azure Database for PostgreSQL y Azure Cache for Redis soportan datos digitales y optimización. La región secundaria permanece en warm standby.

Diagrama de Despliegue - Alta Disponibilidad y Recuperación ante Desastres

Sistema de Banca por Internet BP

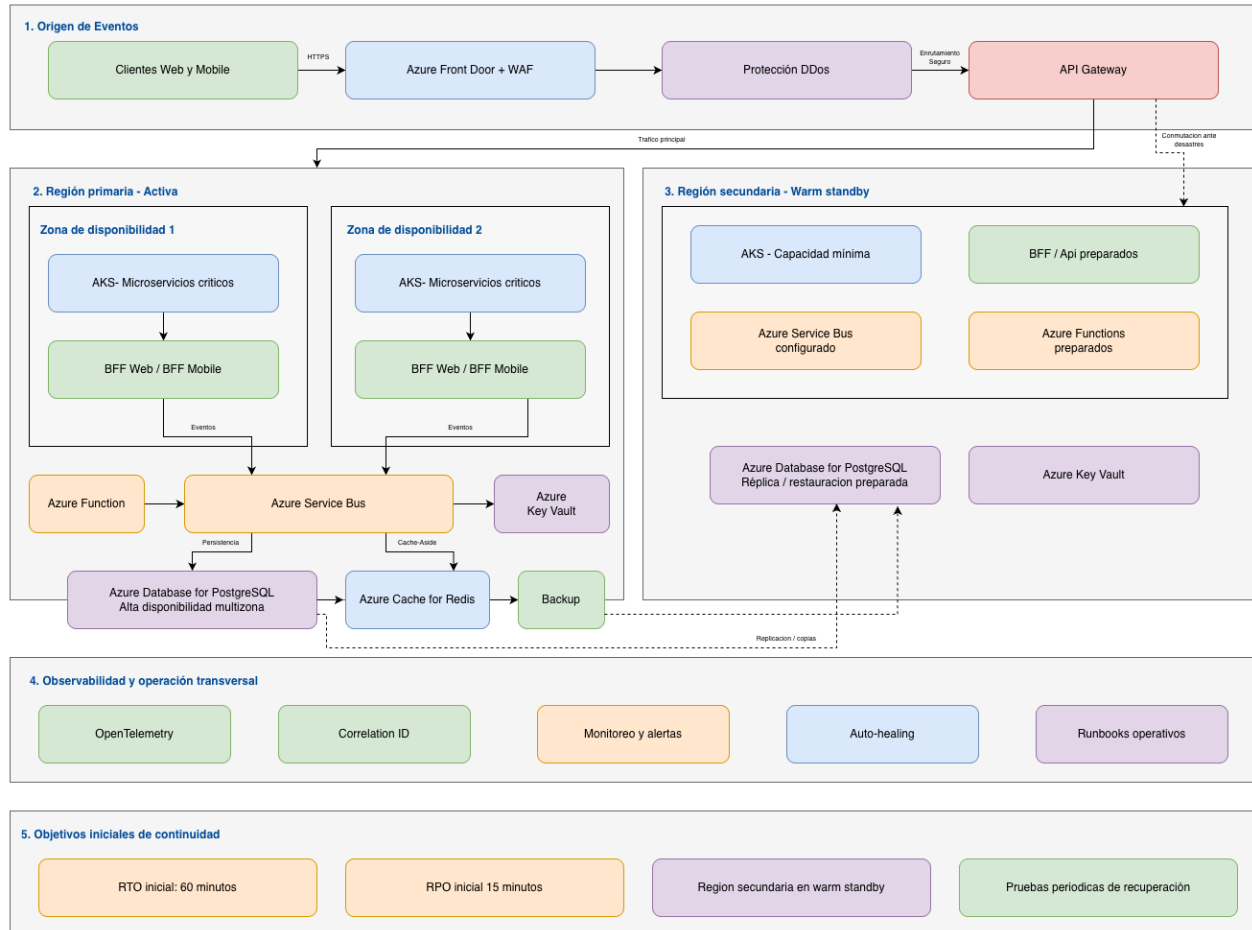


Figura 7. Despliegue en Azure, alta disponibilidad y recuperación ante desastres.

15.2 Azure Kubernetes Service

AKS aloja BFF y microservicios críticos. Se recomienda separar namespaces, identidades, políticas de red, recursos y ciclos de despliegue por ambiente y dominio. La configuración debe considerar nodos distribuidos por zonas, escalado, mantenimiento, seguridad y observabilidad.

Segunda justificación teórica: AKS proporciona programación, escalado y recuperación declarativa para cargas contenedorizadas, permitiendo distribuir réplicas y aplicar despliegues progresivos. También estandariza aislamiento, observabilidad y políticas de plataforma para microservicios con ciclos de cambio independientes.

15.3 Azure Functions

Azure Functions se utiliza para procesos breves, programados o asíncronos, como tareas de publicación, mantenimiento o integración no continua. En consecuencia, no constituye la plataforma principal de microservicios críticos ni debe utilizarse cuando la duración, control o latencia requieran otro modelo.

Segunda justificación teórica: Azure Functions es apropiado para trabajos event-driven porque asigna capacidad en función de ejecuciones y evita mantener procesos ociosos. Esta característica reduce costo y administración en tareas breves, sin trasladar a Functions cargas críticas que requieren control persistente de recursos y latencia.

15.4 Servicios de integración

Azure Service Bus proporciona colas, canales de eventos, suscripciones y dead-letter queue. La Capa Anticorrupción se despliega con alta disponibilidad y conectividad segura hacia sistemas empresariales. La red y los límites de salida deben impedir accesos no autorizados.

Segunda justificación teórica: separar mensajería y adaptación de integración permite absorber variaciones de protocolo, disponibilidad y formato sin contaminar el modelo de negocio. La conectividad restringida y los límites de salida reducen la superficie de exposición hacia sistemas internos y externos.

15.5 Servicios de datos

Azure Database for PostgreSQL utiliza alta disponibilidad multizona en la región primaria y una estrategia de réplica o restauración preparada para la región secundaria. Azure Cache for Redis se dimensiona según carga y criticidad, sin asumir persistencia oficial.

Segunda justificación teórica: utilizar servicios administrados de datos permite delegar tareas repetitivas de respaldo, parcheo y alta disponibilidad, reduciendo riesgo operativo. La separación entre persistencia relacional y caché conserva una fuente durable de verdad para el dominio y una capa independiente de aceleración.

15.6 Servicios de seguridad

Azure Front Door, WAF, protección DDoS, Azure Key Vault, Managed Identities, controles de red y políticas de acceso conforman la base de seguridad cloud. Su configuración debe integrarse con el gobierno corporativo de BP.

15.7 Servicios de observabilidad

La telemetría de aplicaciones, plataformas e integraciones se centraliza para análisis, alertas y tableros. A partir de ahí, openTelemetry permite instrumentación consistente y Correlation ID conecta el recorrido de una solicitud con eventos y operaciones relacionadas.

15.8 Distribución regional y por zonas

La región primaria opera activamente y distribuye cargas entre zonas. La región secundaria warm standby mantiene capacidad mínima y artefactos necesarios. La conmutación debe ser controlada, documentada y validada mediante pruebas.

Segunda justificación teórica: las zonas atienden fallas locales dentro de la región, mientras la región secundaria cubre eventos de alcance regional; son niveles de resiliencia distintos y complementarios. Mantener una estrategia explícita para ambos evita asumir que alta disponibilidad local equivale a recuperación ante desastre.

15.9 Ambientes de desarrollo, pruebas y producción

Desarrollo, pruebas y producción deben estar segregados en identidades, datos, secretos, redes y permisos. Los ambientes no productivos utilizan datos anonimizados o sintéticos y pueden operar con capacidad reducida, sin eliminar controles esenciales.

16. OBSERVABILIDAD Y OPERACIÓN

16.1 Estrategia de observabilidad

La observabilidad permite inferir el estado interno mediante logs, métricas y trazas. Debe cubrir experiencia del cliente, servicios, colas, datos, integraciones y seguridad.

Segunda justificación teórica: observabilidad combina logs, métricas y trazas porque cada señal responde a preguntas diferentes sobre comportamiento, tendencia y recorrido de una solicitud. Esta correlación permite diagnosticar sistemas distribuidos sin depender de reproducir el incidente o ingresar manualmente a cada componente.

16.2 Logs

En el escenario planteado, los logs son estructurados, con niveles, contexto, Correlation ID y exclusión de secretos. Su retención depende de finalidad técnica y política.

16.3 Métricas

Se capturan latencia, tasa de errores, saturación, disponibilidad. Dicho de otra forma, profundidad de colas, tiempos de integración, resultados de transferencias y entregas de notificaciones.

16.4 Trazas distribuidas

En este punto, las trazas conectan canal, BFF, API Gateway, microservicios, mensajería e integraciones, permitiendo identificar cuellos de botella y fallas parciales.

16.5 OpenTelemetry

OpenTelemetry estandariza instrumentación y reduce dependencia de una biblioteca propietaria. La exportación concreta se adapta a la plataforma de monitoreo aprobada.

Segunda justificación teórica: OpenTelemetry estandariza la instrumentación y desacopla el código de un backend específico de monitoreo. Esto facilita conservar continuidad de trazas, métricas y contexto entre lenguajes y componentes, y reduce el costo de cambiar o complementar la plataforma de observabilidad.

16.6 Correlation ID

En términos de diseño, cada solicitud recibe un Correlation ID propagado por llamadas y mensajes. Los identificadores de negocio se mantienen separados para evitar exposición innecesaria.

Segunda justificación teórica: un Correlation ID permite seguir una misma interacción a través de llamadas síncronas y mensajes asíncronos, donde el orden temporal por sí solo no es suficiente. Esta relación acelera análisis de causa y conecta evidencia técnica con la operación de negocio sin exponer identificadores sensibles.

16.7 Alertas

Las alertas se basan en impacto y acción requerida, no solo en umbrales técnicos. En la operación diaria, deben incluir responsable, severidad, contexto y runbook.

16.8 Tableros operativos

Desde una mirada operativa, los tableros muestran salud de canales, transacciones, dependencias, colas, fraude, auditoría, notificaciones y recuperación.

16.9 Monitoreo de experiencia del usuario

Se monitorean disponibilidad percibida, tiempos de carga, errores de canal, abandono y resultados de operaciones, respetando privacidad.

16.10 Gestión de incidentes

El criterio adoptado es el siguiente: la gestión incluye detección, clasificación, comunicación, contención, recuperación, análisis de causa y acciones posteriores. Los incidentes críticos deben preservar evidencia.

17. DEVOPS, CALIDAD Y EXCELENCIA OPERATIVA

17.1 Estrategia de integración y entrega continua

Los cambios se integran frecuentemente, con revisión, compilación, análisis estático, pruebas y promoción controlada. Los pipelines deben ser reproducibles y auditables.

Segunda justificación teórica: integración y entrega continua reducen el tamaño de los cambios y proporcionan retroalimentación temprana mediante pruebas automatizadas. Los pipelines reproducibles disminuyen variabilidad manual y dejan evidencia de qué artefacto, configuración y controles fueron promovidos a cada ambiente.

17.2 Infraestructura como Código

La infraestructura se define como código para consistencia, revisión y recuperación. Este documento recomienda la práctica, pero no entrega Terraform, Bicep ni plantillas implementadas.

Segunda justificación teórica: Infraestructura como Código hace reproducible la configuración, permite revisarla antes de aplicarla y reduce desviaciones entre ambientes. También facilita reconstrucción y recuperación porque la topología deja de depender de cambios manuales difíciles de rastrear.

17.3 Gestión de configuraciones

La configuración se separa del código, se versiona cuando no es sensible y se administra por ambiente. Con ese criterio, los cambios deben ser trazables.

17.4 Gestión de secretos

Los secretos no se almacenan en repositorios ni imágenes. Se obtienen desde Azure Key Vault mediante identidades administradas.

Segunda justificación teórica: separar secretos de repositorios e imágenes evita que una copia del código o del artefacto incluya credenciales reutilizables. La obtención dinámica desde Key Vault permite aplicar rotación y revocación sin distribuir manualmente valores sensibles.

17.5 Estrategia de pruebas

De forma concreta, se incluyen pruebas unitarias, contrato, integración, end-to-end. Accesibilidad, regresión y aceptación de negocio, con mayor profundidad en capacidades críticas.

17.6 Pruebas de seguridad

Se aplican análisis de dependencias, código, imágenes, configuración. Con ese criterio, pruebas dinámicas y ejercicios de penetración según riesgo.

17.7 Pruebas de rendimiento

Se validan carga esperada, picos, degradación, recuperación y límites de dependencias. En consecuencia, midiendo experiencia y capacidad real.

17.8 Pruebas de resiliencia

Se simulan fallas de pods, zonas, dependencias, colas y datos para verificar timeouts, circuit breakers, auto-healing y runbooks. Ese punto debe quedar visible desde el diseño.

Segunda justificación teórica: las propiedades de recuperación no se demuestran únicamente con diseño; deben observarse bajo fallas controladas. Las pruebas de resiliencia validan supuestos sobre timeouts, escalado, auto-healing y runbooks antes de que una falla real exponga comportamientos no previstos.

17.9 Estrategia de despliegue

Aquí conviene precisar que se recomiendan despliegues progresivos, canary o blue-green según criticidad. Con observación y criterios automáticos de detención.

Segunda justificación teórica: canary o blue-green reduce el radio de impacto al exponer una versión nueva de manera progresiva o mantener una versión anterior disponible. La observación de métricas durante la promoción permite detener o revertir el cambio antes de afectar a toda la base de clientes.

17.10 Rollback

Cada cambio debe contar con estrategia de rollback o roll-forward. La decisión busca algo concreto: las migraciones de datos deben ser compatibles y reversibles cuando sea posible.

Segunda justificación teórica: una estrategia explícita de rollback o roll-forward limita el tiempo de exposición ante una versión defectuosa. La compatibilidad de migraciones evita que un cambio de datos haga imposible regresar a una versión estable del servicio.

17.11 Gobierno y control de cambios

Arquitectura, seguridad, datos y operación revisan cambios relevantes. A partir de ahí, las excepciones se documentan con riesgo, responsable y fecha de revisión.

18. REGULACIÓN Y CUMPLIMIENTO

18.1 Consideraciones regulatorias

La implementación debe validar normativa bancaria, seguridad, continuidad, externalización, nube, prevención de fraude y auditoría aplicable a BP. En la operación diaria, este documento no sustituye criterio jurídico o regulatorio.

18.2 Protección de datos personales

Se aplican finalidad, minimización, transparencia, seguridad, acceso restringido y ciclo de vida. Por eso, el tratamiento biométrico requiere evaluación reforzada.

18.3 Trazabilidad de operaciones

Las operaciones deben conservar evidencia suficiente para reconstruir quién, qué, cuándo, desde dónde y con qué resultado, sin almacenar información innecesaria.

18.4 Segregación de funciones

En términos de diseño, desarrollo, operación, seguridad, aprobación y auditoría mantienen roles diferenciados. Los privilegios elevados son temporales, revisados y auditados.

18.5 Retención de registros

Cumplimiento y Legal deben aprobar períodos de retención para auditoría. Dicho de otra forma, operaciones, identidad, fraude, logs y respaldos.

18.6 Gestión de evidencias

Desde una mirada operativa, las evidencias deben conservar integridad, cadena de custodia, acceso controlado, sellos de tiempo y capacidad de exportación autorizada.

18.7 Validaciones pendientes con las áreas de cumplimiento

Deben confirmarse regulación aplicable, residencia, proveedores, transferencias internacionales de datos, consentimiento biométrico, RTO/RPO, retención y requisitos de reporte.

19. COSTOS Y OPTIMIZACIÓN

19.1 Principales generadores de costo

- AKS y nodos de cómputo.
- Azure Database for PostgreSQL y almacenamiento.
- Azure Cache for Redis.
- Azure Service Bus y volumen de mensajes.
- Tráfico, Front Door, WAF y conectividad híbrida.
- Telemetría, retención de logs y respaldos.
- Proveedores biométricos y de notificaciones.
- Capacidad de warm standby y licenciamiento del proveedor de identidad.

19.2 Estrategia de dimensionamiento

El dimensionamiento parte de clientes activos, concurrencia, transacciones por segundo, tamaño de mensajes, retención y objetivos de disponibilidad. En la operación diaria, se recomienda una prueba de capacidad antes de producción y revisión periódica con datos reales.

19.3 Escalabilidad según demanda

La capacidad se ajusta de forma horizontal y controlada. Por eso, los límites deben considerar dependencias que no escalan al mismo ritmo, especialmente Core y red interbancaria. Escalar el canal sin proteger el backend puede trasladar la saturación.

19.4 Optimización de recursos

Se aplican right-sizing, autoscaling, reservas o compromisos cuando exista estabilidad, políticas de retención, apagado de recursos no productivos, optimización de telemetría y revisión de servicios infrautilizados.

19.5 Ambientes no productivos

Desde una mirada operativa, los ambientes no productivos pueden usar capacidad reducida y horarios de operación, manteniendo seguridad y aislamiento. No deben copiar datos productivos sin anonimización y autorización.

19.6 Gobierno financiero cloud

Se asignan etiquetas, presupuestos, alertas, responsables y reportes por ambiente y dominio. En la operación diaria, arquitectura, Operaciones y Finanzas revisan tendencias, anomalías y costo por transacción o cliente activo.

19.7 Consideraciones de crecimiento

El costo crecerá con clientes, transacciones, retención, disponibilidad y canales de notificación. Por eso, la evolución debe preservar modularidad y permitir aislar capacidades costosas sin rediseñar toda la plataforma.

20. RIESGOS ARQUITECTÓNICOS

20.1 Riesgos técnicos

Los principales riesgos técnicos son complejidad operativa de microservicios y AKS. En consecuencia, diseño incorrecto de límites, fallas en consistencia eventual, saturación de dependencias y observabilidad insuficiente.

20.2 Riesgos de integración

La latencia, disponibilidad, contratos no estandarizados y respuestas ambiguas del Core, sistema complementario o red interbancaria pueden afectar la experiencia y consistencia. La Capa Anticorrupción y conciliación reducen, pero no eliminan, estos riesgos.

20.3 Riesgos de seguridad

Para esta propuesta, incluyen robo de credenciales, abuso de APIs, fraude, vulnerabilidades de aplicaciones. Exposición de secretos, configuración incorrecta y tratamiento inadecuado de datos biométricos.

20.4 Riesgos operativos

Incluyen alertas excesivas, runbooks incompletos, dependencia de conocimiento individual, errores de cambio y falta de pruebas de recuperación. La decisión busca algo concreto: deben mitigarse con automatización, entrenamiento y ejercicios.

20.5 Riesgos regulatorios

La ausencia de validación temprana sobre nube, residencia, proveedores, evidencia, consentimiento biométrico y retención puede obligar a cambios tardíos. A partir de ahí, cumplimiento debe participar desde el refinamiento.

20.6 Riesgos de costos

El sobredimensionamiento, telemetría sin control, ambientes permanentes y recuperación regional pueden elevar costos. La reducción excesiva también puede comprometer disponibilidad y RTO/RPO.

20.7 Acciones de mitigación

Riesgo	Impacto	Mitigación	Responsable sugerido
Contratos limitados del Core	Alto	Pruebas tempranas, Capa Anticorrupción, timeouts y conciliación.	Arquitectura e Integración
Duplicidad transaccional	Alto	Idempotency Key, Transactional Outbox y consumidores idempotentes.	Desarrollo
Falla regional	Alto	Warm standby, respaldos, runbooks y pruebas de recuperación.	Plataforma y Operaciones
Fraude o toma de cuenta	Alto	MFA, step-up, antifraude, monitoreo y respuesta.	Seguridad y Riesgos
Complejidad de AKS	Medio/Alto	Plataforma común, automatización, SRE y límites pragmáticos.	Plataforma
Costos no controlados	Medio	Presupuestos, etiquetas, right-sizing y revisión mensual.	FinOps
Incumplimiento regulatorio	Alto	Validación temprana y evidencia de controles.	Cumplimiento y Legal

21. ROADMAP DE IMPLEMENTACIÓN

21.1 Criterios de priorización

La priorización considera valor al cliente, riesgo, dependencia, cumplimiento, capacidad de integración y aprendizaje. La seguridad, identidad, observabilidad y base operativa deben establecerse antes de escalar funcionalidades transaccionales.

21.2 Producto mínimo viable

- Considerar de forma explícita base cloud, seguridad perimetral, identidad corporativa y observabilidad mínima.
- SPA y aplicación móvil con BFF y API Gateway.
- Consulta de perfil, productos y movimientos.
- Mantener como criterio: transferencias entre cuentas propias con idempotencia, auditoría y notificaciones.
- Onboarding inicial con proveedor biométrico si es requisito de salida.
- Operación multizona, respaldos y runbooks esenciales.

21.3 Primera fase de evolución

- Transferencias interbancarias y conciliación ampliada.
- Pagos y gestión de beneficiarios.
- Reglas de límites y antifraude más avanzadas.
- Mayor automatización de despliegues, pruebas y respuesta operativa.
- Ejercicio completo de recuperación regional.

21.4 Segunda fase de evolución

- Optimización de consultas mediante CQRS ligero y caché basada en mediciones.
- Personalización de experiencia y preferencias de notificación.
- Mantener como criterio: capacidades avanzadas de análisis de fraude y monitoreo de experiencia.
- Revisión de aislamiento de servicios y capacidad según crecimiento.

21.5 Capacidades futuras

- Passkeys/FIDO2 como evolución de autenticación.
- Mayor automatización de recuperación y pruebas de caos controladas.
- Nuevos productos y canales reutilizando APIs y servicios existentes.
- Evaluación de escenarios regionales más exigentes si el negocio requiere menor RTO.

21.6 Dependencias y condiciones de implementación

El roadmap depende de contratos de integración, disponibilidad de ambientes, capacidades del proveedor de identidad, selección biométrica, aprobación regulatoria, conectividad, datos de prueba, equipo operativo y presupuesto. Con ese criterio, cada fase debe cerrar con criterios medibles de seguridad, rendimiento, resiliencia y aceptación de negocio.

Fase	Resultado principal	Condición de salida
Fundaciones	Plataforma, identidad, seguridad y observabilidad.	Controles aprobados y despliegue repetible.
MVP	Consultas y transferencias propias operativas.	Pruebas funcionales, seguridad, carga y recuperación aprobadas.
Evolución 1	Interbancarias, pagos y conciliación.	Integraciones estables y operación entrenada.
Evolución 2	Optimización y capacidades avanzadas.	Métricas reales justifican inversiones y separación adicional.

22. CONCLUSIONES

22.1 Cumplimiento de los objetivos

Para esta propuesta, la arquitectura propuesta cubre canales web y móvil, identidad corporativa. Onboarding biométrico, consultas, transferencias, pagos, notificaciones, auditoría, integración con Core y sistema complementario, alta disponibilidad, tolerancia a fallos, recuperación, seguridad, monitoreo, auto-healing y excelencia operativa.

22.2 Beneficios de la arquitectura

La separación entre canales, dominios e integración reduce acoplamiento y protege al negocio frente a cambios de sistemas existentes. La decisión busca algo concreto: los patrones de idempotencia, Saga, Transactional Outbox y conciliación controlan riesgos transaccionales. La observabilidad y resiliencia permiten operar con mayor previsibilidad.

22.3 Decisiones que requieren validación

Deben validarse volúmenes, SLAs de sistemas empresariales, proveedor de identidad, proveedor biométrico, topología de red, RTO/RPO, regulación, residencia, retención, costos y capacidad del equipo. En consecuencia, estas validaciones pueden ajustar dimensionamiento o secuencia, sin alterar los principios centrales.

22.4 Recomendación final

Se recomienda implementar de forma incremental, iniciando por fundaciones de seguridad, identidad, integración, observabilidad y operación, y avanzando hacia capacidades transaccionales priorizadas. El enfoque de microservicios debe mantenerse pragmático: aislar donde exista valor comprobable y evitar fragmentación innecesaria. El Core debe conservar en todo momento su rol de sistema oficial de registro financiero.